

Python 程式語言

Lecture 11: Data Visualization

Instructor: 顏家珩 Chia-Heng Yen

Course Roadmap

- Fundamentals & Environment
 - Introduction to Python and Development Environment Setup
 - Basic Syntax and Flow Control
 - Data Structures and Collections
- Advanced Programming
 - Modular and Functional Programming
 - File Handling and Exception Handling
 - Object-Oriented Programming
 - Algorithm Implementation in Python
- User Interface Development
 - GUI Programming with Tkinter and PyQt
- Data Processing & Analysis
 - Data Analysis Fundamentals
 - Scientific Computing Applications
 - Data Visualization
- AI & ML Applications
 - Introduction to Machine Learning with Applications
 - AI-Related Library in Python Applications

Course Objectives

- Master Matplotlib for creating various chart types
- Use Seaborn for statistical visualizations
- Create interactive visualizations with Plotly

Outline

- Matplotlib Fundamentals
- Advanced Matplotlib Techniques
- Seaborn Statistical Graphics
- Interactive Visualization with Plotly
- Advanced Plotly & Dashboards

Matplotlib Fundamentals

What is Matplotlib?

- Matplotlib is the most widely used plotting library in Python
- Key Features
 - Comprehensive 2D plotting
 - Highly customizable
 - Publication-quality figures
 - Supports multiple output formats
- Installation
 - `$ pip install matplotlib`
 - `$ conda install matplotlib`



Matplotlib Architecture

- Two Main Interfaces

```
import matplotlib.pyplot as plt
import numpy as np
```

Method 1: pyplot interface

Quick and simple for basic plots

```
plt.plot([1, 2, 3, 4], [1, 4, 9, 16])
```

```
plt.show()
```

Method 2: Object-oriented interface

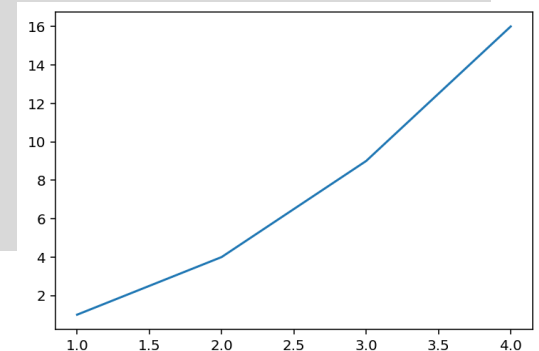
More control for complex visualizations

```
fig, ax = plt.subplots()
```

```
ax.plot([1, 2, 3, 4], [1, 4, 9, 16])
```

```
plt.show()
```

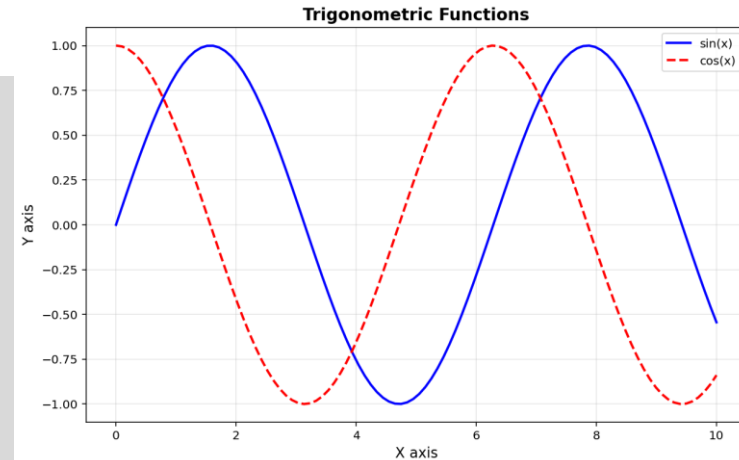
- When to use each?
 - pyplot: Quick exploration
 - OO interface: Complex layouts, multiple subplots



Basic Plot Types

- Line Plot

```
import matplotlib.pyplot as plt
import numpy as np
# Generate data
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)
# Create line plot
plt.figure(figsize=(10, 6))
plt.plot(x, y1, label='sin(x)', linewidth=2, color='blue')
plt.plot(x, y2, label='cos(x)', linewidth=2, color='red', linestyle='--')
# Add labels and title
plt.xlabel('X axis', fontsize=12)
plt.ylabel('Y axis', fontsize=12)
plt.title('Trigonometric Functions', fontsize=14, fontweight='bold')
plt.legend(loc='upper right')
plt.grid(True, alpha=0.3)
plt.show()
```



- Use cases: Time series, trends, continuous data

Basic Plot Types

- Scatter Plot

```
# Generate random data
```

```
np.random.seed(42)
```

```
n_points = 100
```

```
x = np.random.randn(n_points)
```

```
y = 2 * x + np.random.randn(n_points) * 0.5
```

```
colors = np.random.rand(n_points)
```

```
sizes = 1000 * np.random.rand(n_points)
```

```
# Create scatter plot
```

```
plt.figure(figsize=(10, 6))
```

```
scatter = plt.scatter(x, y, c=colors, s=sizes, alpha=0.6, cmap='viridis', edgecolors='black',  
linewidth=1)
```

```
plt.xlabel('X Variable', fontsize=12)
```

```
plt.ylabel('Y Variable', fontsize=12)
```

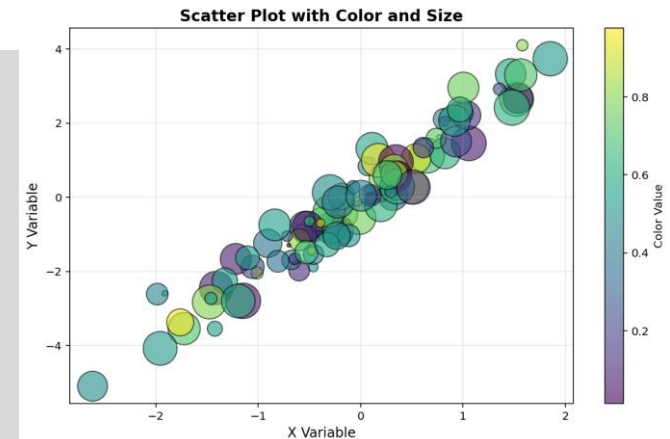
```
plt.title('Scatter Plot with Color and Size', fontsize=14, fontweight='bold')
```

```
# Add colorbar
```

```
plt.colorbar(scatter, label='Color Value')
```

```
plt.grid(True, alpha=0.3)
```

```
plt.show()
```



- Use cases: Correlation, distribution, clustering

Basic Plot Types

- Bar Chart

Sample data

```
categories = ['Product A', 'Product B', 'Product C', 'Product D']
```

```
sales_2023 = [45, 67, 89, 56]
```

```
sales_2024 = [52, 71, 95, 63]
```

Position of bars

```
x = np.arange(len(categories))
```

```
width = 0.35
```

Create bar chart

```
fig, ax = plt.subplots(figsize=(10, 6))
```

```
bars1 = ax.bar(x - width/2, sales_2023, width, label='2023', color='skyblue', edgecolor='black')
```

```
bars2 = ax.bar(x + width/2, sales_2024, width, label='2024', color='lightcoral', edgecolor='black')
```

Add value labels on bars

```
for bars in [bars1, bars2]:
```

```
    for bar in bars:
```

```
        height = bar.get_height()
```

```
        ax.text(bar.get_x() + bar.get_width()/2., height, f'{height}', ha='center', va='bottom', fontsize=10)
```

```
ax.set_xlabel('Products', fontsize=12)
```

```
ax.set_ylabel('Sales (units)', fontsize=12)
```

```
ax.set_title('Product Sales Comparison', fontsize=14, fontweight='bold')
```

```
ax.set_xticks(x)
```

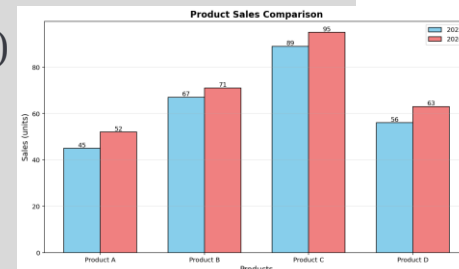
```
ax.set_xticklabels(categories)
```

```
ax.legend()
```

```
ax.grid(axis='y', alpha=0.3)
```

```
plt.tight_layout()
```

```
plt.show()
```



Basic Plot Types

- Histogram

```
# Generate sample data
```

```
np.random.seed(42)
```

```
data1 = np.random.normal(100, 15, 1000) # Mean 100, SD 15
```

```
data2 = np.random.normal(110, 20, 1000) # Mean 110, SD 20
```

```
# Create histogram
```

```
plt.figure(figsize=(10, 6))
```

```
plt.hist(data1, bins=30, alpha=0.6, label='Group A', color='blue', edgecolor='black')
```

```
plt.hist(data2, bins=30, alpha=0.6, label='Group B', color='red', edgecolor='black')
```

```
plt.xlabel('Value', fontsize=12)
```

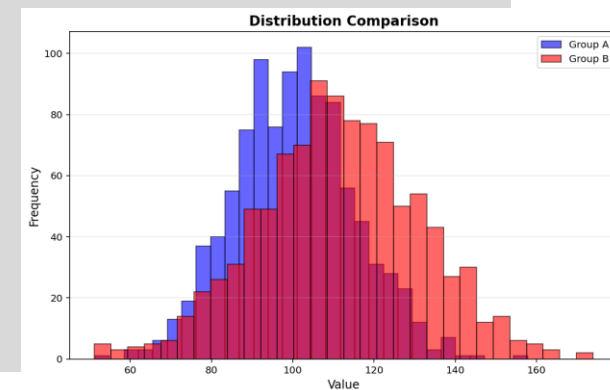
```
plt.ylabel('Frequency', fontsize=12)
```

```
plt.title('Distribution Comparison', fontsize=14, fontweight='bold')
```

```
plt.legend()
```

```
plt.grid(axis='y', alpha=0.3)
```

```
plt.show()
```



- Use cases: Distribution analysis, frequency, data ranges

Basic Plot Types

- Pie Chart

```
# Market share data
```

```
labels = ['Company A', 'Company B', 'Company C', 'Company D', 'Others']
```

```
sizes = [35, 25, 20, 12, 8]
```

```
colors = ['#ff9999', '#66b3ff', '#99ff99', '#ffcc99', '#ff99cc']
```

```
explode = (0.1, 0, 0, 0, 0) # Explode first slice
```

```
# Create pie chart
```

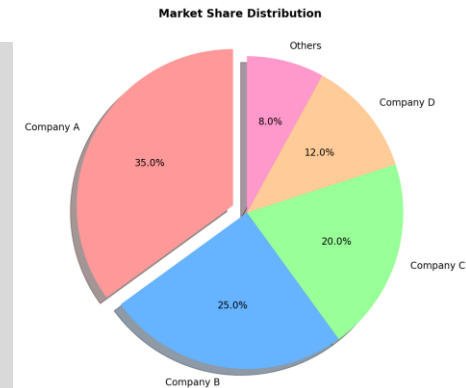
```
plt.figure(figsize=(10, 8))
```

```
plt.pie(sizes, explode=explode, labels=labels, colors=colors, autopct='%1.1f%%',  
shadow=True, startangle=90, textprops={'fontsize': 12})
```

```
plt.title('Market Share Distribution', fontsize=14, fontweight='bold', pad=20)
```

```
plt.axis('equal') # Equal aspect ratio ensures circular pie
```

```
plt.show()
```



- Best Practice: Use for 5-7 categories maximum

Subplots and Multiple Figures

- Creating Multiple Subplots

```
# Prepare data
```

```
x = np.linspace(0, 10, 100)
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
```

```
# Plot 1: Line plot
```

```
axes[0, 0].plot(x, np.sin(x), 'b-', linewidth=2)
axes[0, 0].set_title('Sine Wave',
fontweight='bold')
axes[0, 0].grid(True, alpha=0.3)
```

```
# Plot 2: Scatter plot
```

```
axes[0, 1].scatter(x, np.cos(x), c=x,
cmap='viridis')
axes[0, 1].set_title('Cosine Scatter',
fontweight='bold')
axes[0, 1].grid(True, alpha=0.3)
```

```
# Plot 3: Bar chart
```

```
categories = ['A', 'B', 'C', 'D']
values = [4, 7, 2, 5]
axes[1, 0].bar(categories, values,
color='lightgreen', edgecolor='black')
axes[1, 0].set_title('Bar Chart',
fontweight='bold')
axes[1, 0].grid(axis='y', alpha=0.3)
```

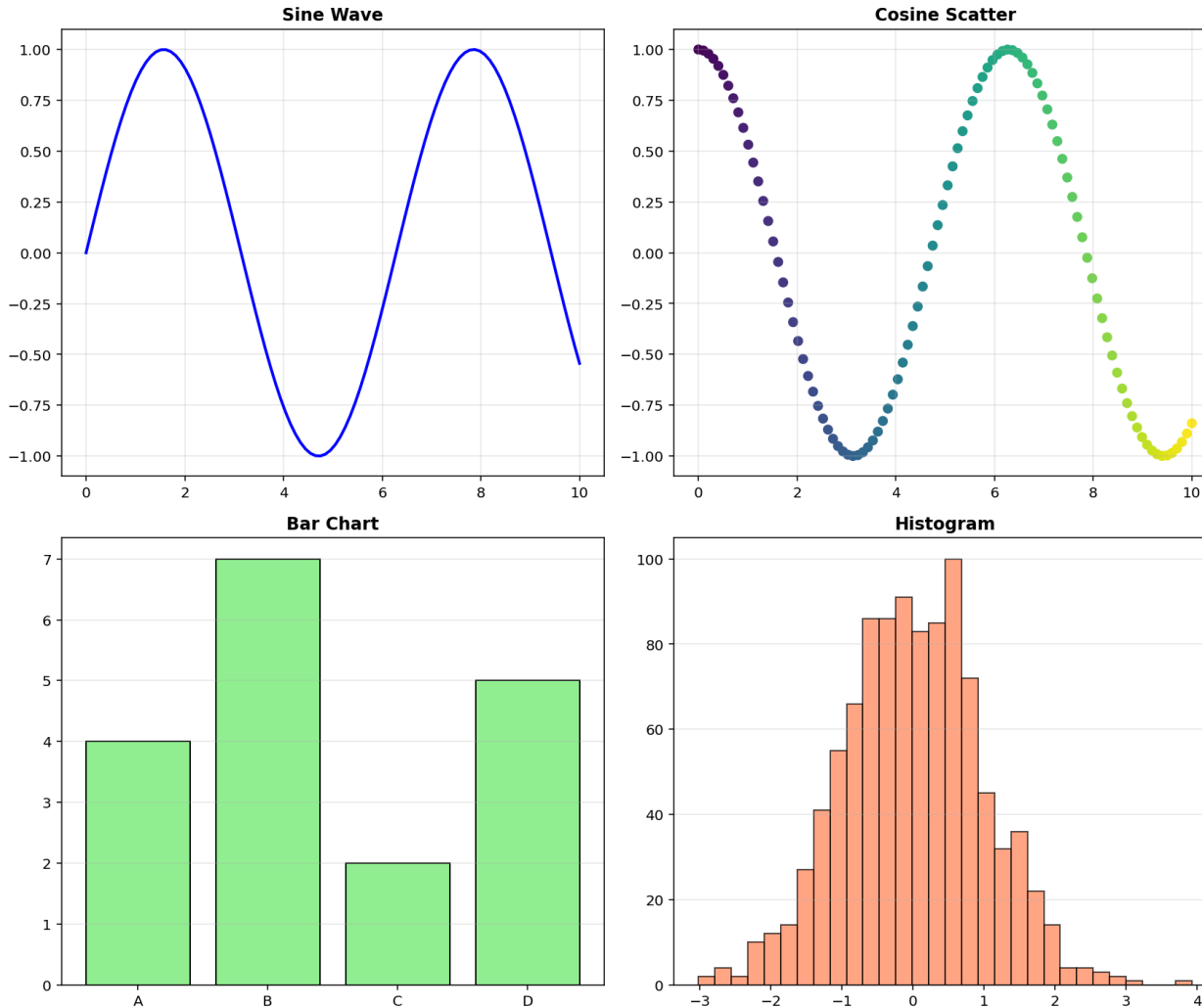
```
# Plot 4: Histogram
```

```
data = np.random.randn(1000)
axes[1, 1].hist(data, bins=30, color='coral',
edgecolor='black', alpha=0.7)
axes[1, 1].set_title('Histogram',
fontweight='bold')
axes[1, 1].grid(axis='y', alpha=0.3)
```

```
# Adjust layout
```

```
plt.tight_layout()
plt.show()
```

Subplots and Multiple Figures



Customizing Plots

- Colors, Styles, and Markers

Sample data

```
x = np.linspace(0, 10, 50)
```

```
plt.figure(figsize=(12, 6))
```

Different line styles

```
plt.plot(x, x, 'r-', label='Solid', linewidth=2)
```

```
plt.plot(x, x+1, 'g--', label='Dashed', linewidth=2)
```

```
plt.plot(x, x+2, 'b-.', label='Dash-dot', linewidth=2)
```

```
plt.plot(x, x+3, 'm:', label='Dotted', linewidth=2)
```

With markers

```
plt.plot(x[::5], (x+4)[::5], 'ko-', label='Circle markers', markersize=8, linewidth=2)
```

```
plt.plot(x[::5], (x+5)[::5], 'rs-', label='Square markers', markersize=8, linewidth=2)
```

```
plt.xlabel('X axis', fontsize=12)
```

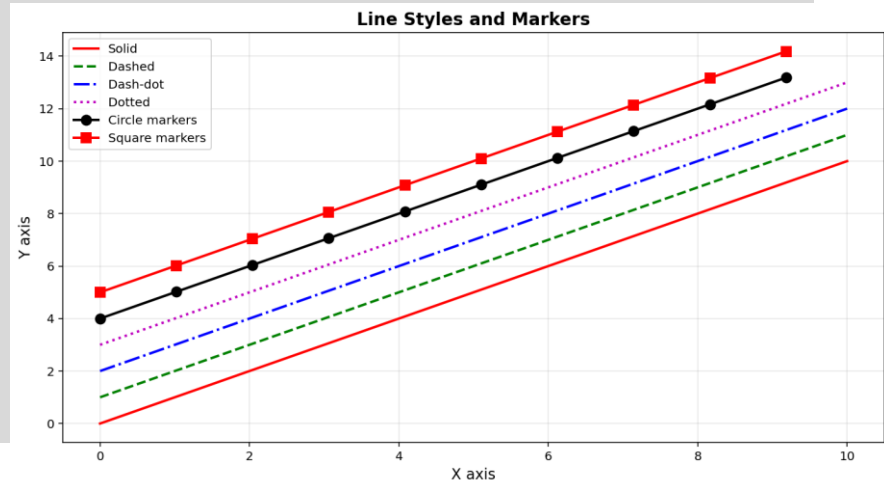
```
plt.ylabel('Y axis', fontsize=12)
```

```
plt.title('Line Styles and Markers',  
         fontsize=14, fontweight='bold')
```

```
plt.legend(loc='upper left', fontsize=10)
```

```
plt.grid(True, alpha=0.3)
```

```
plt.show()
```



Advanced Customization

- Annotations and Text

```
# Sample data
```

```
x = np.linspace(0, 10, 100)
```

```
y = np.sin(x)
```

```
fig, ax = plt.subplots(figsize=(12, 6))
```

```
ax.plot(x, y, 'b-', linewidth=2)
```

```
# Add annotations
```

```
# Maximum point
```

```
max_idx = np.argmax(y)
```

```
ax.annotate('Maximum', xy=(x[max_idx],
```

```
y[max_idx]), xytext=(x[max_idx]+1,
```

```
y[max_idx]+0.3),
```

```
arrowprops=dict(arrowstyle='->',
```

```
color='red',
```

```
lw=2), fontsize=12, color='red',
```

```
fontweight='bold')
```

```
# Minimum point
```

```
min_idx = np.argmin(y)
```

```
ax.annotate('Minimum', xy=(x[min_idx],
```

```
y[min_idx]), xytext=(x[min_idx]+1,
```

```
y[min_idx]-0.3),
```

```
arrowprops=dict(arrowstyle='->',
```

```
color='green', lw=2), fontsize=12,
```

```
color='green', fontweight='bold')
```

```
# Add text box
```

```
textstr = 'y = sin(x)\nPeriod: 2 $\pi$ '
```

```
props = dict(boxstyle='round',
```

```
facecolor='wheat', alpha=0.5)
```

```
ax.text(0.05, 0.95, textstr,
```

```
transform=ax.transAxes, fontsize=12,
```

```
verticalalignment='top', bbox=props)
```

```
ax.set_xlabel('X axis', fontsize=12)
```

```
ax.set_ylabel('Y axis', fontsize=12)
```

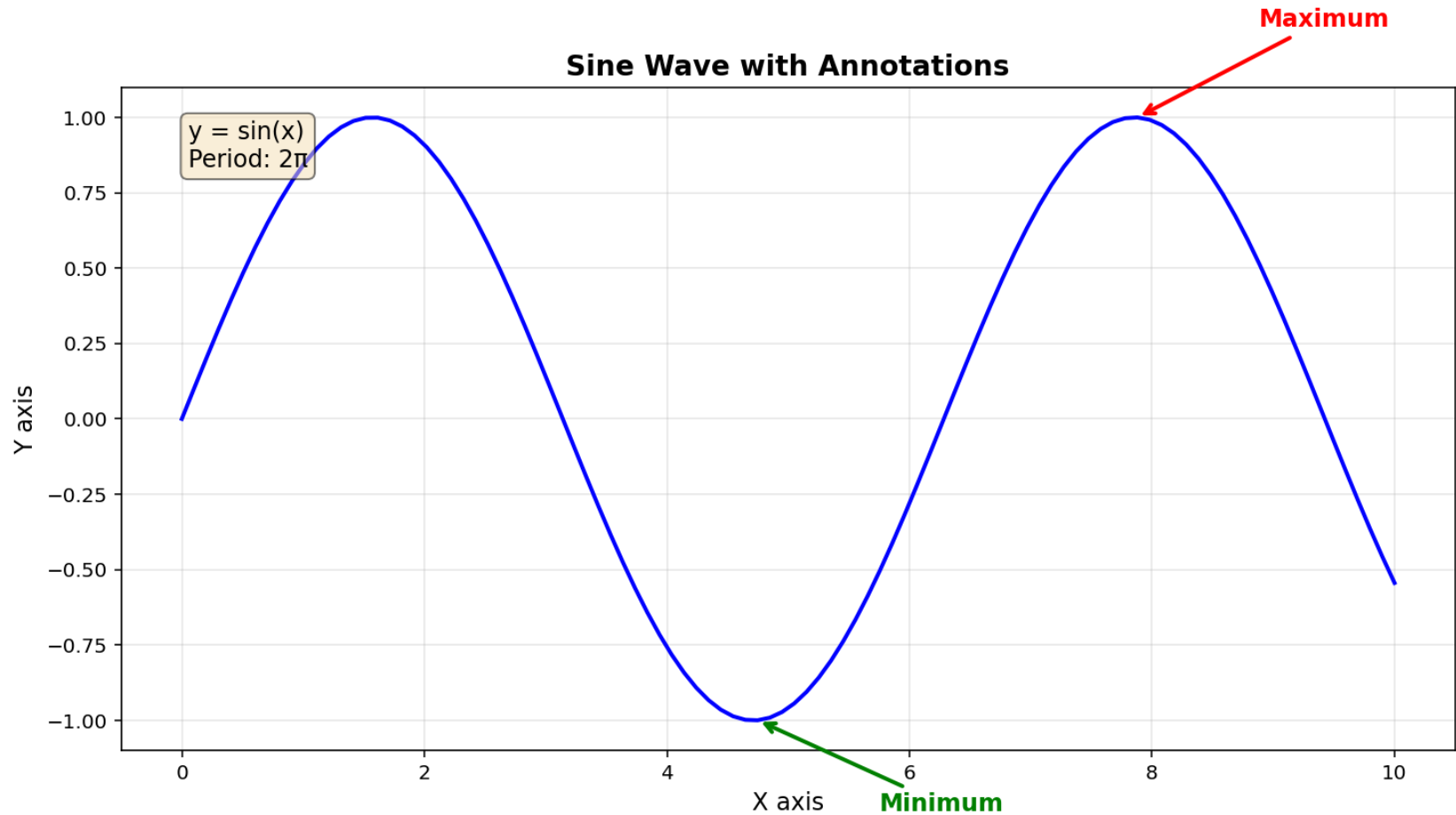
```
ax.set_title('Sine Wave with Annotations',
```

```
fontsize=14, fontweight='bold')
```

```
ax.grid(True, alpha=0.3)
```

```
plt.show()
```


Advanced Customization



Saving Figures

```
# Create a sample plot
plt.figure(figsize=(10, 6))
x = np.linspace(0, 10, 100)
plt.plot(x, np.sin(x), 'b-', linewidth=2, label='sin(x)')
plt.plot(x, np.cos(x), 'r--', linewidth=2, label='cos(x)')
plt.xlabel('X axis', fontsize=12)
plt.ylabel('Y axis', fontsize=12)
plt.title('Trigonometric Functions', fontsize=14, fontweight='bold')
plt.legend()
plt.grid(True, alpha=0.3)

# Save in different formats
plt.savefig('plot.png', dpi=300, bbox_inches='tight')      # PNG format
plt.savefig('plot.pdf', bbox_inches='tight')               # PDF format
plt.savefig('plot.svg', bbox_inches='tight')               # SVG format
print("Figures saved successfully!")
```

- Saving Options
 - dpi: Resolution (default 100, use 300 for publication)
 - bbox_inches='tight': Remove extra whitespace
 - transparent=True: Transparent background

Advanced Matplotlib Techniques

Complex Subplot Layouts

- GridSpec for Custom Layouts

```
import matplotlib.gridspec as gridspec
# Create figure with GridSpec
fig = plt.figure(figsize=(14, 10))
gs = gridspec.GridSpec(3, 3, figure=fig,
hspace=0.3, wspace=0.3)
# Large plot spanning multiple cells
ax1 = fig.add_subplot(gs[0:2, 0:2])
x = np.linspace(0, 10, 100)
ax1.plot(x, np.sin(x), 'b-', linewidth=2)
ax1.set_title('Main Plot - Sine Wave',
fontweight='bold', fontsize=12)
ax1.grid(True, alpha=0.3)

# Small plots
ax2 = fig.add_subplot(gs[0, 2])
ax2.hist(np.random.randn(1000), bins=20,
color='coral', edgecolor='black')
ax2.set_title('Distribution', fontsize=10)

ax3 = fig.add_subplot(gs[1, 2])
ax3.scatter(np.random.randn(50),
np.random.randn(50), alpha=0.6)
ax3.set_title('Scatter', fontsize=10)

ax4 = fig.add_subplot(gs[2, :])
categories = ['A', 'B', 'C', 'D', 'E']
values = [23, 45, 56, 78, 32]
ax4.bar(categories, values, color='lightgreen',
edgecolor='black')
ax4.set_title('Bar Chart', fontweight='bold',
fontsize=10)
ax4.grid(axis='y', alpha=0.3)

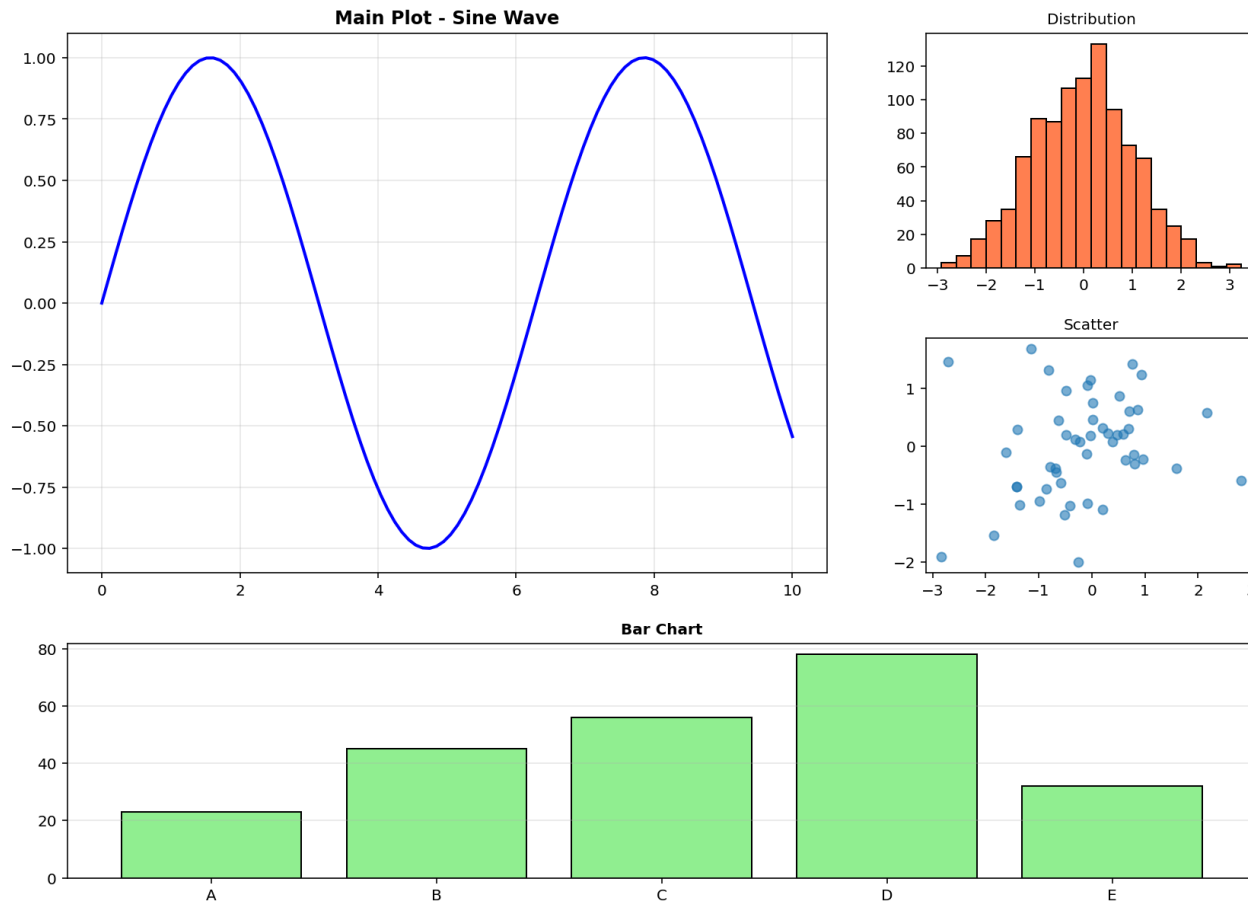
plt.suptitle('Complex Layout with GridSpec ',
fontsize=16, fontweight='bold', y=0.98)

plt.show()
```

Complex Subplot Layouts

- GridSpec for Custom Layouts

Complex Layout with GridSpec



Twin Axes

```
# Create data with different scales
x = np.linspace(0, 10, 100)
y1 = np.sin(x) # Range: -1 to 1
y2 = np.exp(x/3) # Range: 1 to 28

fig, ax1 = plt.subplots(figsize=(12, 6))

# First y-axis
color = 'tab:blue'
ax1.set_xlabel('X axis', fontsize=12)
ax1.set_ylabel('Sine Wave', color=color,
               fontsize=12)
line1 = ax1.plot(x, y1, color=color,
                 linewidth=2, label='sin(x)')
ax1.tick_params(axis='y', labelcolor=color)
ax1.grid(True, alpha=0.3)
```

```
# Second y-axis
ax2 = ax1.twinx()
color = 'tab:red'
ax2.set_ylabel('Exponential', color=color,
               fontsize=12)
line2 = ax2.plot(x, y2, color=color,
                 linewidth=2, linestyle='--', label='exp(x/3)')
ax2.tick_params(axis='y', labelcolor=color)

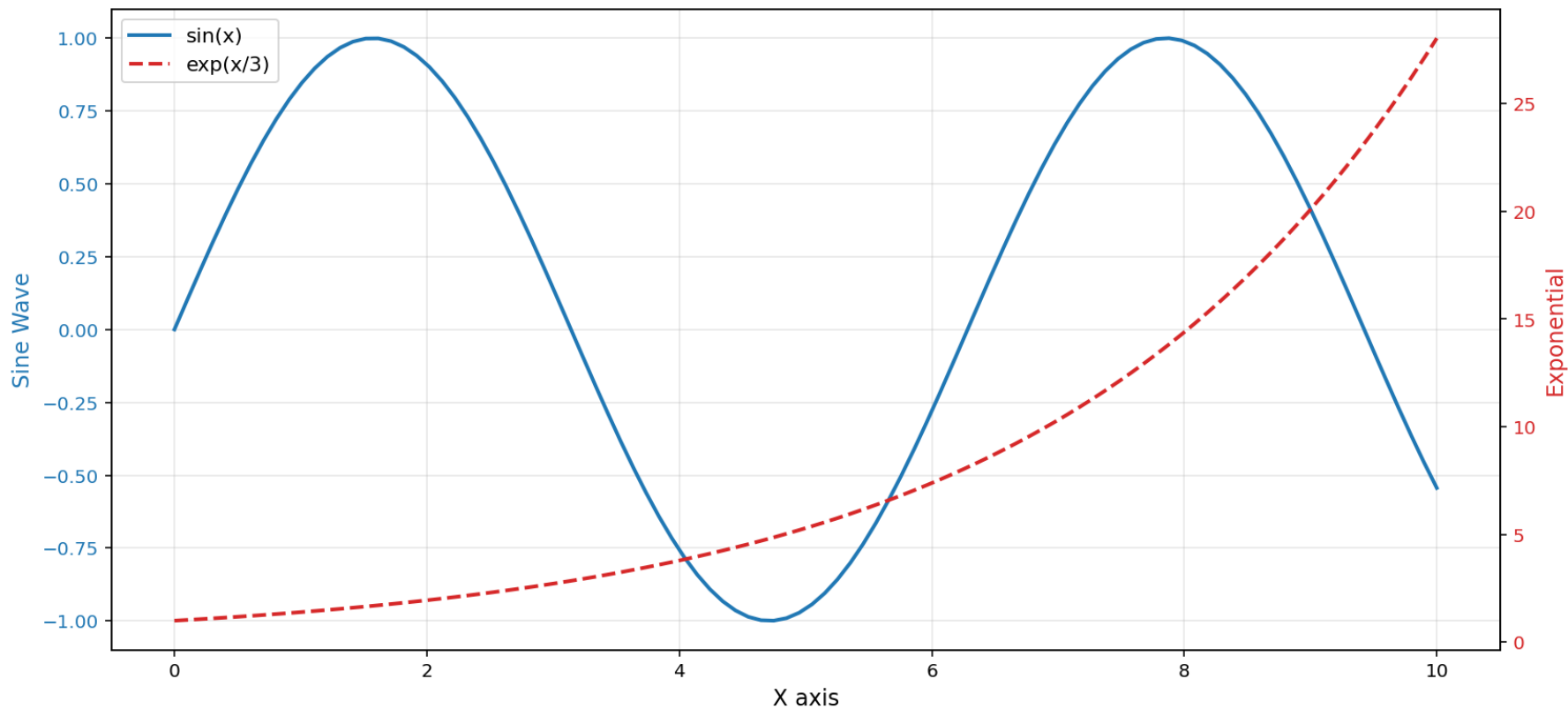
# Combined legend
lines = line1 + line2
labels = [l.get_label() for l in lines]
ax1.legend(lines, labels, loc='upper left',
           fontsize=11)

plt.title('Twin Axes - Different Scales',
          fontsize=14, fontweight='bold', pad=20)
plt.tight_layout()
plt.show()
```

- Use case: Displaying data with different units or scales

Twin Axes

Twin Axes - Different Scales



Inset Plots

Create main plot

```
fig, ax = plt.subplots(figsize=(12, 8))
```

```
x = np.linspace(0, 10, 1000)
```

```
y = np.sin(x) * np.exp(-x/10)
```

```
ax.plot(x, y, 'b-', linewidth=2)
```

```
ax.set_xlabel('X axis', fontsize=12)
```

```
ax.set_ylabel('Y axis', fontsize=12)
```

```
ax.set_title('Main Plot with Inset', fontsize=14,  
fontweight='bold')
```

```
ax.grid(True, alpha=0.3)
```

Add inset plot

```
from mpl_toolkits.axes_grid1.inset_locator  
import inset_axes
```

```
axins = inset_axes(ax, width="40%",  
height="40%", loc='upper right',  
bbox_to_anchor=(0, 0, 1, 1),  
bbox_transform=ax.transAxes)
```

Zoom into a specific region

```
x_zoom = x[(x >= 0) & (x <= 2)]
```

```
y_zoom = y[(x >= 0) & (x <= 2)]
```

```
axins.plot(x_zoom, y_zoom, 'r-', linewidth=2)
```

```
axins.set_title('Zoomed Region', fontsize=10)
```

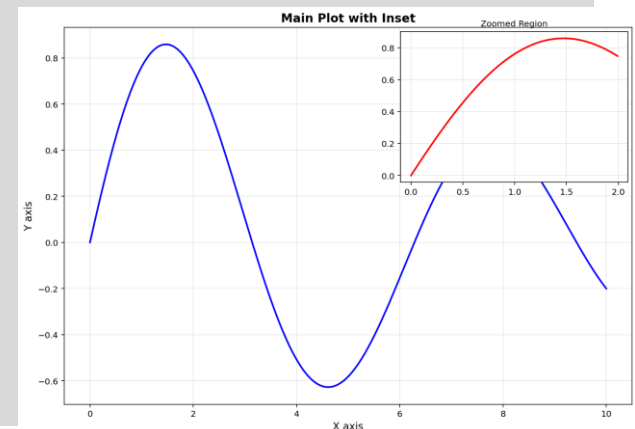
```
axins.grid(True, alpha=0.3)
```

Mark the zoomed region on main plot

```
from matplotlib.patches import Rectangle  
rect = Rectangle((0, -0.1), 2, 1.2, linewidth=2,  
edgecolor='red', facecolor='none', linestyle='—')  
)
```

```
ax.add_patch(rect)
```

```
plt.show()
```



Contour and 3D Plots

- Contour Plot

```
# Generate 2D data
```

```
x = np.linspace(-3, 3, 100)
```

```
y = np.linspace(-3, 3, 100)
```

```
X, Y = np.meshgrid(x, y)
```

```
Z = np.sin(np.sqrt(X**2 + Y**2))
```

```
# Create contour plot
```

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
```

```
# Filled contour
```

```
contourf = ax1.contourf(X, Y, Z, levels=20, cmap='viridis')
```

```
ax1.set_title('Filled Contour Plot', fontweight='bold', fontsize=12)
```

```
ax1.set_xlabel('X axis', fontsize=11)
```

```
ax1.set_ylabel('Y axis', fontsize=11)
```

```
fig.colorbar(contourf, ax=ax1, label='Z value')
```

```
# Line contour with labels
```

```
contour = ax2.contour(X, Y, Z, levels=15, colors='black', linewidths=1)
```

```
ax2.clabel(contour, inline=True, fontsize=8)
```

```
contourf2 = ax2.contourf(X, Y, Z, levels=15, cmap='coolwarm', alpha=0.6)
```

```
ax2.set_title('Contour with Labels', fontweight='bold', fontsize=12)
```

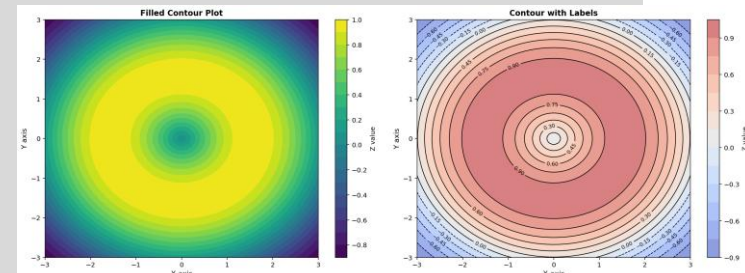
```
ax2.set_xlabel('X axis', fontsize=11)
```

```
ax2.set_ylabel('Y axis', fontsize=11)
```

```
fig.colorbar(contourf2, ax=ax2, label='Z value')
```

```
plt.tight_layout()
```

```
plt.show()
```



Contour and 3D Plots

- 3D Surface Plot

```
from mpl_toolkits.mplot3d import Axes3D
```

```
# Generate 3D data
```

```
x = np.linspace(-5, 5, 50)
```

```
y = np.linspace(-5, 5, 50)
```

```
X, Y = np.meshgrid(x, y)
```

```
Z = np.sin(np.sqrt(X**2 + Y**2))
```

```
# Create 3D plot
```

```
fig = plt.figure(figsize=(14, 6))
```

```
# Surface plot
```

```
ax1 = fig.add_subplot(121, projection='3d')
```

```
surf = ax1.plot_surface(X, Y, Z, cmap='viridis',  
alpha=0.9)
```

```
ax1.set_xlabel('X axis', fontsize=10)
```

```
ax1.set_ylabel('Y axis', fontsize=10)
```

```
ax1.set_zlabel('Z axis', fontsize=10)
```

```
ax1.set_title('3D Surface Plot',  
fontweight='bold', fontsize=12)
```

```
fig.colorbar(surf, ax=ax1, shrink=0.5, aspect=5)
```

```
# Wireframe plot
```

```
ax2 = fig.add_subplot(122, projection='3d')
```

```
ax2.plot_wireframe(X, Y, Z, color='blue',  
linewidth=0.5, alpha=0.7)
```

```
ax2.set_xlabel('X axis', fontsize=10)
```

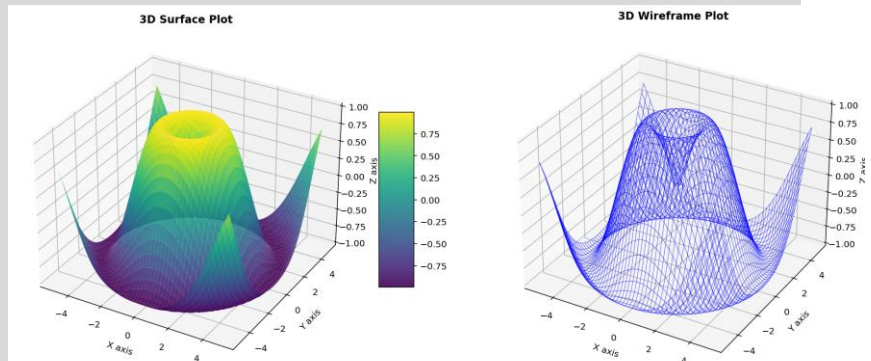
```
ax2.set_ylabel('Y axis', fontsize=10)
```

```
ax2.set_zlabel('Z axis', fontsize=10)
```

```
ax2.set_title('3D Wireframe Plot',  
fontweight='bold', fontsize=12)
```

```
plt.tight_layout()
```

```
plt.show()
```



Animation with Matplotlib

```
from matplotlib.animation import
FuncAnimation

# Create figure and axis
fig, ax = plt.subplots(figsize=(10, 6))
x = np.linspace(0, 2*np.pi, 100)
line, = ax.plot(x, np.sin(x), 'b-', linewidth=2)

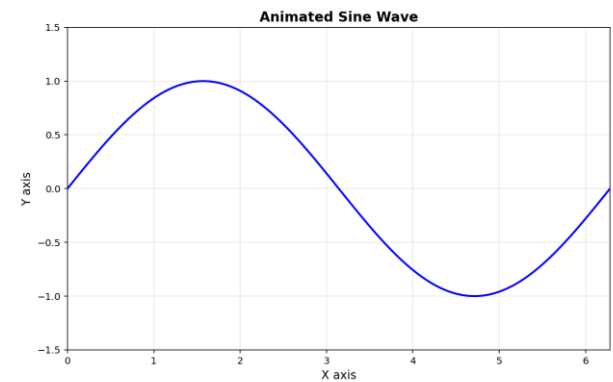
ax.set_xlim(0, 2*np.pi)
ax.set_ylim(-1.5, 1.5)
ax.set_xlabel('X axis', fontsize=12)
ax.set_ylabel('Y axis', fontsize=12)
ax.set_title('Animated Sine Wave',
            fontsize=14, fontweight='bold')
ax.grid(True, alpha=0.3)
```

```
# Animation function
def animate(frame):
    line.set_ydata(np.sin(x + frame/10))
    return line,

# Create animation
anim = FuncAnimation(fig, animate,
                    frames=100, interval=50, blit=True)

# Save animation (optional)
# anim.save('sine_wave.gif', writer='pillow',
#          fps=20)
```

```
plt.show()
```



- Note: Animation requires display; use `anim.save()` for files

Polar Plots (極座標圖)

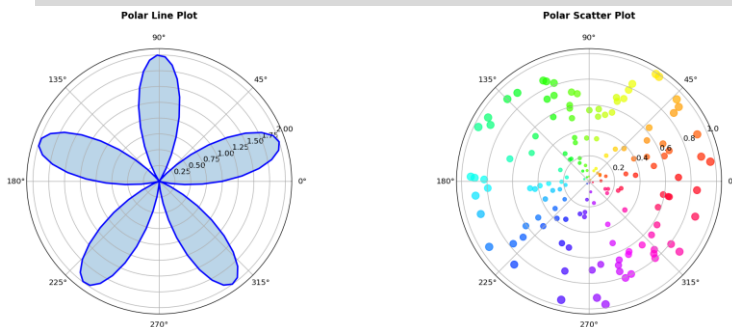
```
# Create polar plot
```

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6), subplot_kw=dict(projection='polar'))
```

```
# Polar line plot
```

```
theta = np.linspace(0, 2*np.pi, 100)  
r = 1 + np.sin(5*theta)
```

```
ax1.plot(theta, r, 'b-', linewidth=2)  
ax1.fill(theta, r, alpha=0.3)  
ax1.set_title('Polar Line Plot',  
fontweight='bold', fontsize=12, pad=20)  
ax1.grid(True)
```



```
# Polar scatter plot
```

```
np.random.seed(42)  
n = 150  
theta2 = 2 * np.pi * np.random.rand(n)  
r2 = np.random.rand(n)  
colors = theta2  
sizes = 100 * r2
```

```
ax2.scatter(theta2, r2, c=colors, s=sizes,  
cmap='hsv', alpha=0.7)  
ax2.set_title('Polar Scatter Plot',  
fontweight='bold', fontsize=12, pad=20)  
ax2.grid(True)
```

```
plt.tight_layout()  
plt.show()
```

- Use cases: Circular data, directional data, radar charts

Error Bars and Confidence Intervals

```
# Generate data with errors
```

```
x = np.linspace(0, 10, 20)
y = 2 * x + 5
y_error = np.random.rand(20) * 3
x_error = np.random.rand(20) * 0.5
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
```

```
# Error bars
```

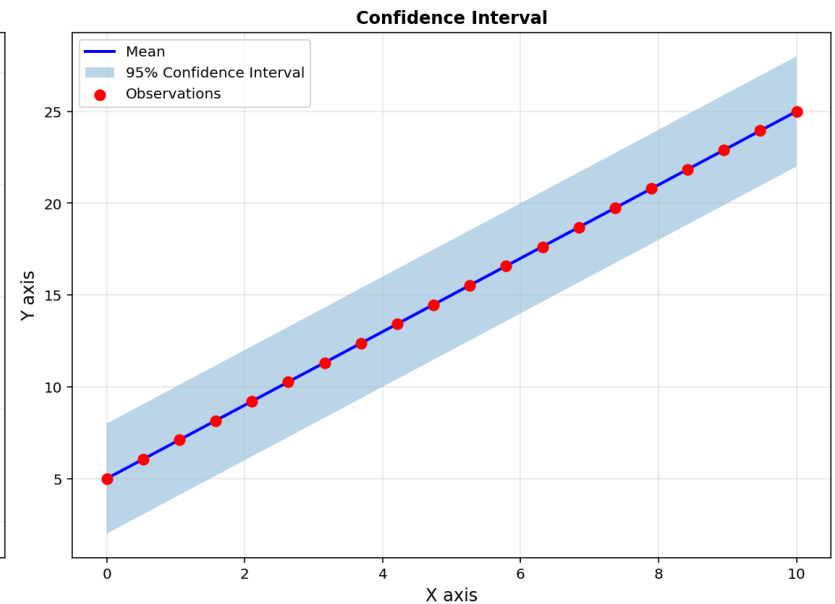
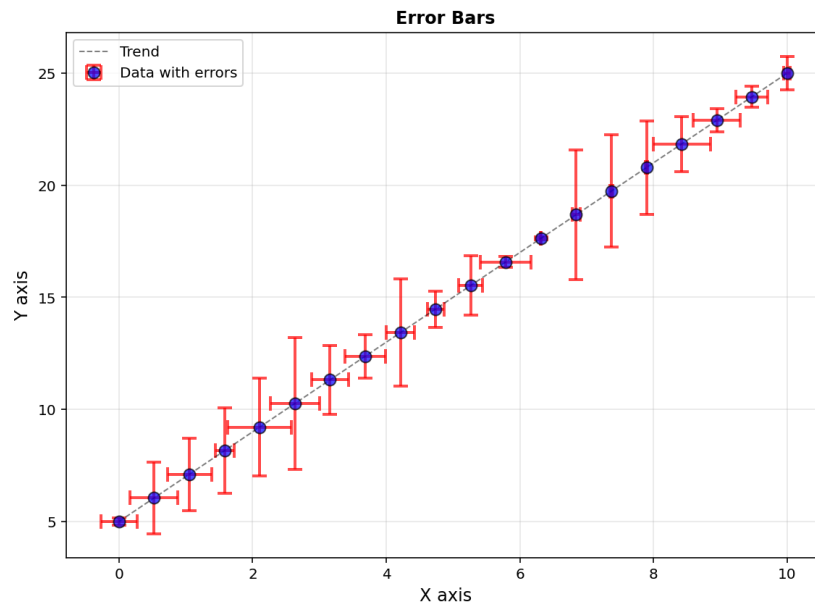
```
ax1.errorbar(x, y, yerr=y_error, xerr=x_error,
fmt='o', markersize=8, capsize=5, capthick=2,
ecolor='red', elinewidth=2,
markerfacecolor='blue', markeredgecolor='black',
alpha=0.7, label='Data with errors')
ax1.plot(x, y, 'k--', linewidth=1, alpha=0.5,
label='Trend')
ax1.set_xlabel('X axis', fontsize=12)
ax1.set_ylabel('Y axis', fontsize=12)
ax1.set_title('Error Bars', fontweight='bold',
fontsize=12)
ax1.legend(fontsize=10)
ax1.grid(True, alpha=0.3)
```

```
# Confidence interval
```

```
x_smooth = np.linspace(0, 10, 100)
y_smooth = 2 * x_smooth + 5
confidence = 3

ax2.plot(x_smooth, y_smooth, 'b-',
linewidth=2, label='Mean')
ax2.fill_between(x_smooth, y_smooth -
confidence, y_smooth + confidence,
alpha=0.3, label='95% Confidence
Interval')
ax2.scatter(x, y, color='red', s=50, zorder=5,
label='Observations')
ax2.set_xlabel('X axis', fontsize=12)
ax2.set_ylabel('Y axis', fontsize=12)
ax2.set_title('Confidence Interval',
fontweight='bold', fontsize=12)
ax2.legend(fontsize=10)
ax2.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Error Bars and Confidence Intervals



Custom Colormaps and Styling

```
from matplotlib.colors import
LinearSegmentedColormap

# Create custom colormap
colors = ['darkblue', 'blue', 'cyan', 'yellow',
'orange', 'red', 'darkred']
n_bins = 100
cmap_name = 'custom_colormap'
custom_cmap =
LinearSegmentedColormap.from_list(
cmap_name, colors, N=n_bins)

# Generate data
x = np.linspace(-3, 3, 100)
y = np.linspace(-3, 3, 100)
X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X**2 + Y**2)) * np.cos(X)
* np.sin(Y)

# Create plot with custom colormap
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
```

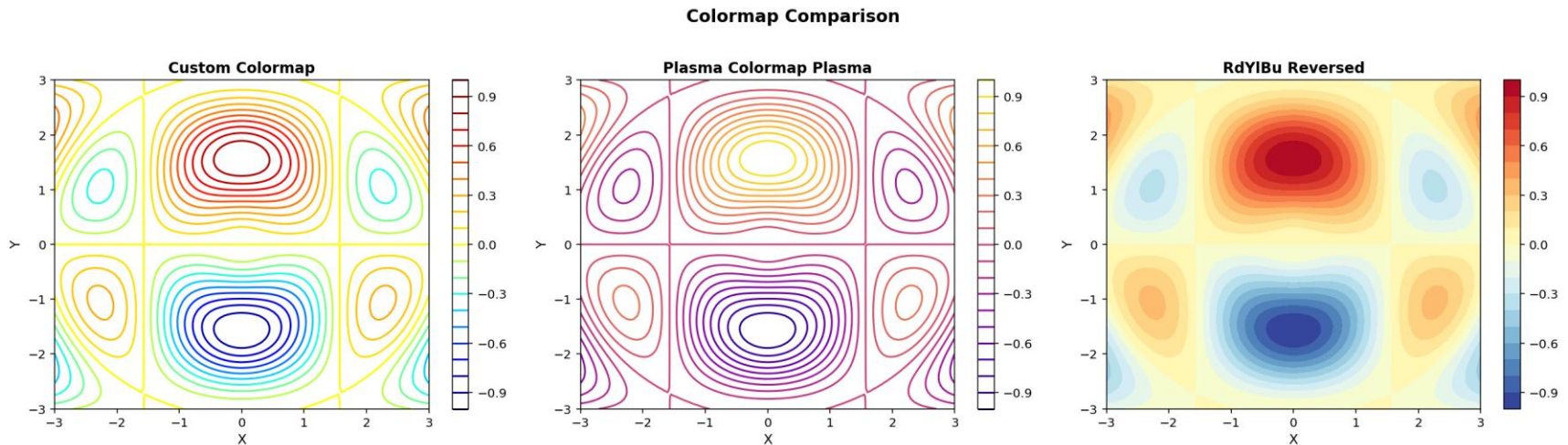
```
# Custom colormap
im1 = axes[0].contour(X, Y, Z, levels=20,
cmap=custom_cmap)
axes[0].set_title('Custom Colormap',
fontweight='bold', fontsize=12)
fig.colorbar(im1, ax=axes[0])

# Built-in colormap 1
im2 = axes[1].contour(X, Y, Z, levels=20,
cmap='plasma')
axes[1].set_title('Plasma
Colormap Plasma', fontweight='bold',
fontsize=12)
fig.colorbar(im2, ax=axes[1])

# Built-in colormap 2
im3 = axes[2].contourf(X, Y, Z, levels=20,
cmap='RdYlBu_r')
axes[2].set_title('RdYlBu Reversed',
fontweight='bold', fontsize=12)
fig.colorbar(im3, ax=axes[2])
```

Custom Colormaps and Styling

```
for ax in axes:  
    ax.set_xlabel('X', fontsize=11)  
    ax.set_ylabel('Y', fontsize=11)  
  
plt.suptitle('Colormap Comparison', fontsize=14, fontweight='bold', y=1.02)  
plt.tight_layout()  
plt.show()
```



Seaborn Statistical Graphics

What is Seaborn?

- Seaborn is a statistical data visualization library built on Matplotlib
- Key Advantages
 - Built-in statistical plots
 - Beautiful default themes
 - Better for exploring relationships
 - Integrates well with Pandas
- Installation
 - \$ pip install seaborn
 - \$ conda install seaborn



```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

```
# Set style
sns.set_style("whitegrid")
sns.set_palette("husl")
```

Seaborn Themes and Styles

Available styles

```
styles = ['darkgrid', 'whitegrid', 'dark', 'white', 'ticks']
```

```
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
```

```
axes = axes.flatten()
```

```
x = np.linspace(0, 10, 100)
```

```
y = np.sin(x)
```

```
for idx, style in enumerate(styles):
```

```
    sns.set_style(style)
```

```
    axes[idx].plot(x, y, linewidth=2)
```

```
    axes[idx].set_title(f'Style: {style}', fontweight='bold')
```

```
    axes[idx].set_xlabel('X')
```

```
    axes[idx].set_ylabel('Y')
```

Hide extra subplot

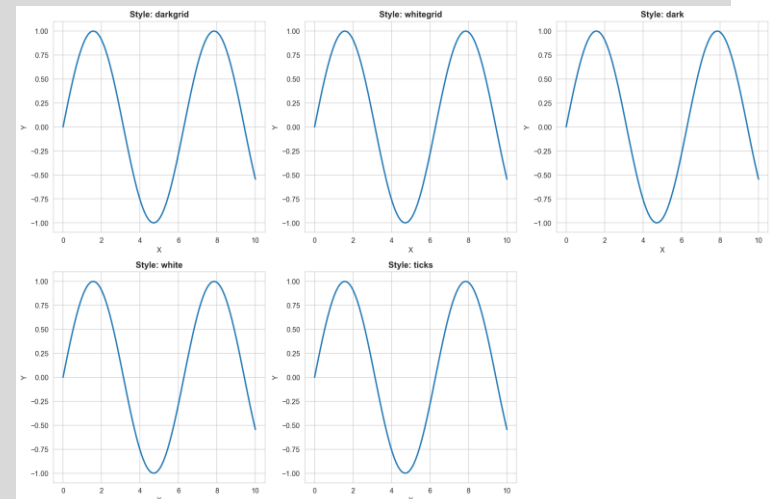
```
axes[5].axis('off')
```

```
plt.tight_layout()
```

```
plt.show()
```

Reset to default

```
sns.set_style("whitegrid")
```



Distribution Plots

- Histogram with KDE (Kernel density estimation)

```
# Generate sample data
```

```
np.random.seed(42)
```

```
data = np.random.normal(100, 15, 1000)
```

```
# Create distribution plot
```

```
plt.figure(figsize=(12, 6))
```

```
# Using histplot
```

```
sns.histplot(data, bins=30, kde=True, color='skyblue', edgecolor='black', linewidth=1.2)
```

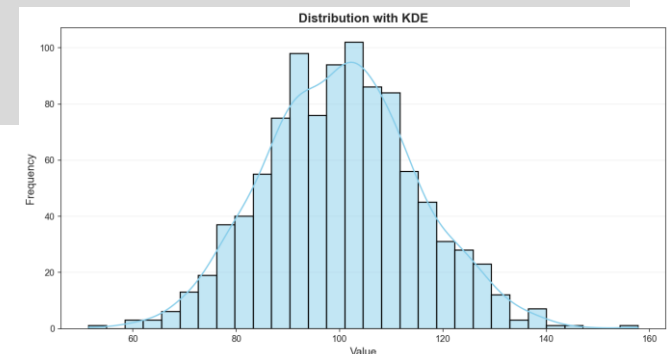
```
plt.xlabel('Value', fontsize=12)
```

```
plt.ylabel('Frequency', fontsize=12)
```

```
plt.title('Distribution with KDE', fontsize=14, fontweight='bold')
```

```
plt.grid(axis='y', alpha=0.3)
```

```
plt.show()
```



Distribution Plots

- KDE Plot

Multiple distributions

```
plt.figure(figsize=(12, 6))
```

```
data1 = np.random.normal(100, 15, 1000)
```

```
data2 = np.random.normal(110, 20, 1000)
```

```
data3 = np.random.normal(90, 10, 1000)
```

```
sns.kdeplot(data=data1, label='Group A', linewidth=2.5, fill=True, alpha=0.3)
```

```
sns.kdeplot(data=data2, label='Group B', linewidth=2.5, fill=True, alpha=0.3)
```

```
sns.kdeplot(data=data3, label='Group C', linewidth=2.5, fill=True, alpha=0.3)
```

```
plt.xlabel('Value', fontsize=12)
```

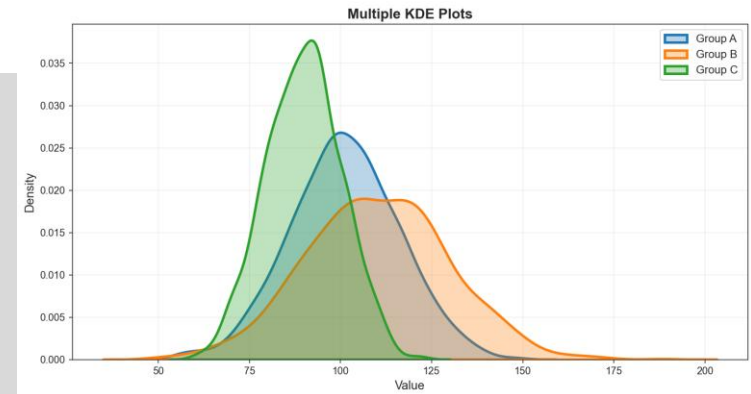
```
plt.ylabel('Density', fontsize=12)
```

```
plt.title('Multiple KDE Plots', fontsize=14, fontweight='bold')
```

```
plt.legend(fontsize=11)
```

```
plt.grid(alpha=0.3)
```

```
plt.show()
```



Categorical Plots

- Box Plot

Create sample dataset

```
np.random.seed(42)
```

```
categories = ['Product A', 'Product B', 'Product C', 'Product D']
```

```
data_dict = {
```

```
    'Category': np.repeat(categories, 100),
```

```
    'Value': np.concatenate([  
        np.random.normal(80, 10, 100),  
        np.random.normal(90, 15, 100),  
        np.random.normal(75, 8, 100),  
        np.random.normal(95, 12, 100)
```

```
    ]) }
```

```
df = pd.DataFrame(data_dict)
```

Create box plot

```
plt.figure(figsize=(12, 6))
```

```
sns.boxplot(data=df, x='Category', y='Value', palette='Set2', linewidth=2)
```

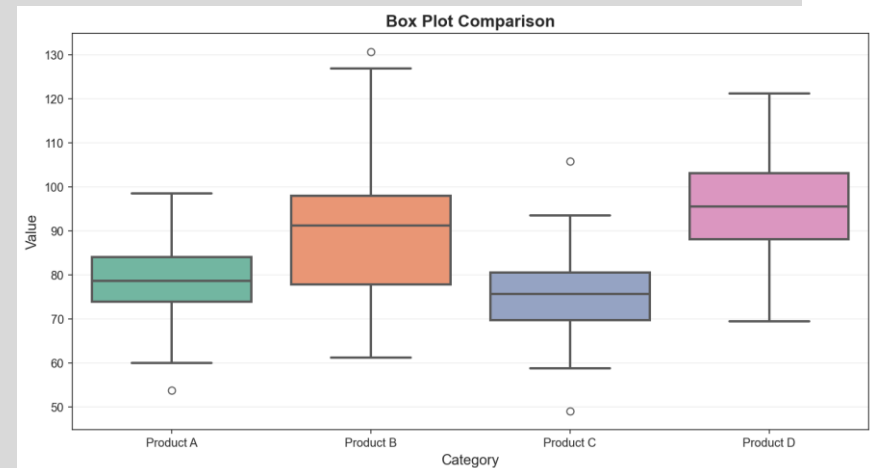
```
plt.xlabel('Category', fontsize=12)
```

```
plt.ylabel('Value', fontsize=12)
```

```
plt.title('Box Plot Comparison', fontsize=14, fontweight='bold')
```

```
plt.grid(axis='y', alpha=0.3)
```

```
plt.show()
```



Categorical Plots

- Violin Plot

Violin plot shows distribution shape

```
plt.figure(figsize=(12, 6))
```

```
sns.violinplot(data=df, x='Category', y='Value', palette='muted', linewidth=2)
```

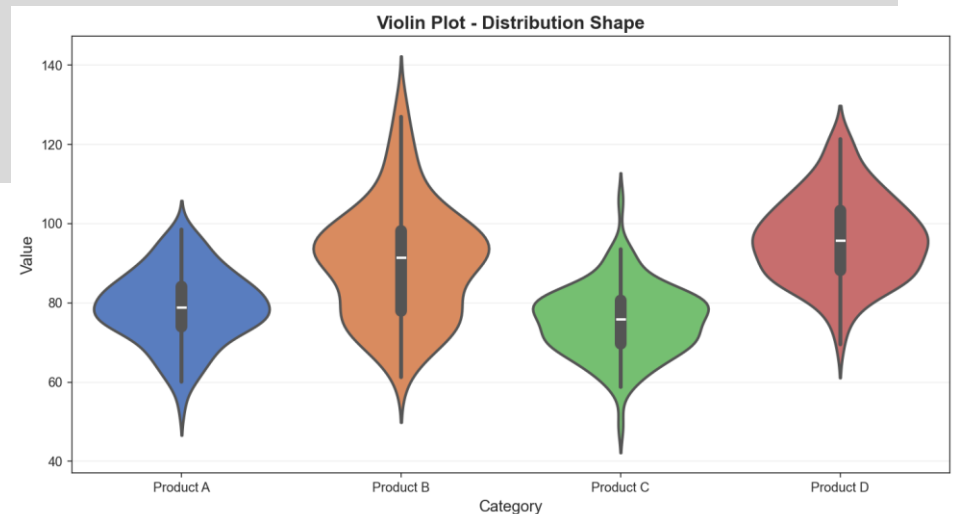
```
plt.xlabel('Category', fontsize=12)
```

```
plt.ylabel('Value', fontsize=12)
```

```
plt.title('Violin Plot - Distribution Shape', fontsize=14, fontweight='bold')
```

```
plt.grid(axis='y', alpha=0.3)
```

```
plt.show()
```



Relationship Plots

- Scatter Plot with Regression

```
# Generate correlated data
```

```
np.random.seed(42)
```

```
n = 100
```

```
x = np.random.randn(n)
```

```
y = 2 * x + np.random.randn(n) * 0.5
```

```
category = np.random.choice(['A', 'B', 'C'], n)
```

```
data_df = pd.DataFrame({'X': x, 'Y': y, 'Category': category})
```

```
# Create scatter plot with regression
```

```
plt.figure(figsize=(12, 6))
```

```
sns.scatterplot(data=data_df, x='X', y='Y', hue='Category', style='Category', s=100, alpha=0.7)
```

```
sns.regplot(data=data_df, x='X', y='Y', scatter=False, color='red', line_kws={'linewidth': 2, 'linestyle': '--'})
```

```
plt.xlabel('X Variable', fontsize=12)
```

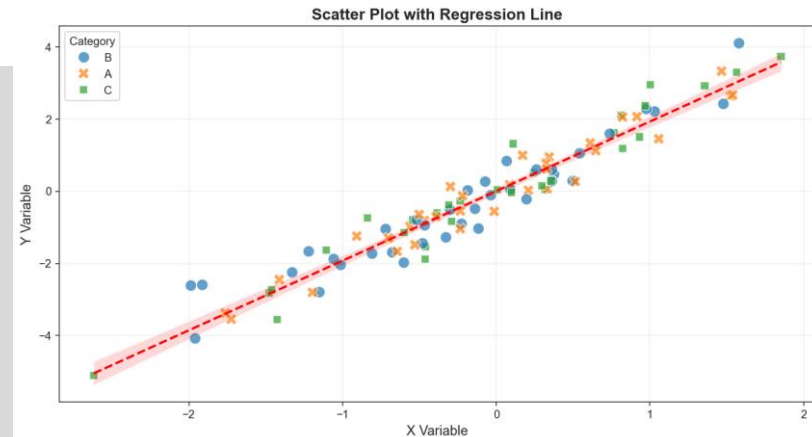
```
plt.ylabel('Y Variable', fontsize=12)
```

```
plt.title('Scatter Plot with Regression Line', fontsize=14, fontweight='bold')
```

```
plt.legend(title='Category', fontsize=10)
```

```
plt.grid(alpha=0.3)
```

```
plt.show()
```



Heatmap

```
# Create correlation matrix
```

```
np.random.seed(42)
data = np.random.randn(10, 12)
df_heatmap = pd.DataFrame(
    data,
    columns=[f'Feature {i+1}' for i in range(12)],
    index=[f'Sample {i+1}' for i in range(10)])
```

```
correlation_matrix = df_heatmap.corr()
```

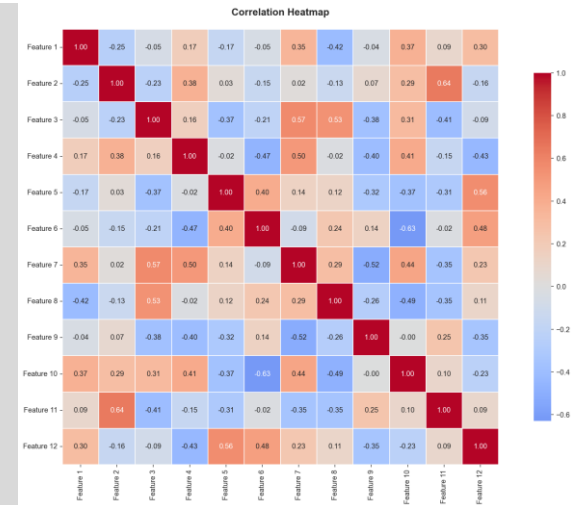
```
# Create heatmap
```

```
plt.figure(figsize=(14, 10))
sns.heatmap(correlation_matrix, annot=True, fmt='.2f', cmap='coolwarm', center=0,
            square=True, linewidths=1, cbar_kws={"shrink": 0.8})
```

```
plt.title('Correlation Heatmap', fontsize=14, fontweight='bold', pad=20)
```

```
plt.tight_layout()
```

```
plt.show()
```



- Use cases: Correlation analysis, feature relationships

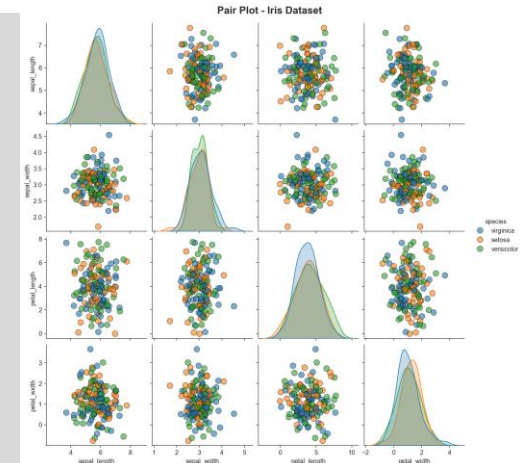
Pair Plot

```
# Load sample dataset
# Create sample data similar to iris dataset
np.random.seed(42)
n_samples = 150
iris_data = pd.DataFrame({
    'sepal_length': np.random.normal(5.8, 0.8, n_samples),
    'sepal_width': np.random.normal(3.0, 0.4, n_samples),
    'petal_length': np.random.normal(3.8, 1.8, n_samples),
    'petal_width': np.random.normal(1.2, 0.8, n_samples),
    'species': np.random.choice(['setosa', 'versicolor', 'virginica'], n_samples)
})

# Create pair plot
sns.pairplot(iris_data, hue='species', diag_kind='kde', plot_kws={'alpha': 0.6, 's': 80,
'edgecolor': 'k'}, height=2.5)

plt.suptitle('Pair Plot - Iris Dataset', y=1.02, fontsize=16, fontweight='bold')

plt.show()
```



- Purpose: Explore relationships between all variable pairs

Joint Plot

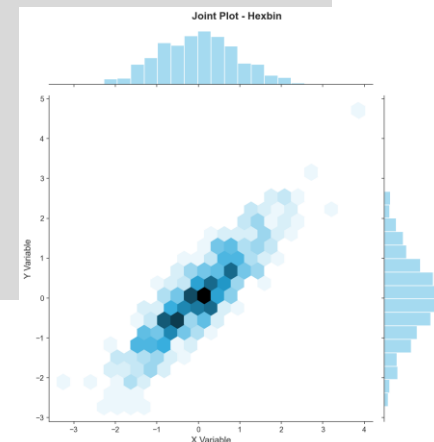
```
# Generate data
np.random.seed(42)
x = np.random.randn(500)
y = x + np.random.randn(500) * 0.5

# Create joint plot
g = sns.jointplot(x=x, y=y, kind='scatter', height=8, marginal_kws=dict(bins=30, fill=True))
g.set_axis_labels('X Variable', 'Y Variable', fontsize=12)
g.fig.suptitle('Joint Plot with Marginal Distributions', y=1.02, fontsize=14, fontweight='bold')

plt.show()

# Alternative: hexbin for large datasets
g2 = sns.jointplot(x=x, y=y, kind='hex', height=8, color='skyblue')
g2.set_axis_labels('X Variable', 'Y Variable', fontsize=12)
g2.fig.suptitle('Joint Plot - Hexbin', y=1.02, fontsize=14, fontweight='bold')

plt.show()
```



FacetGrid

Create sample data

```
np.random.seed(42)
```

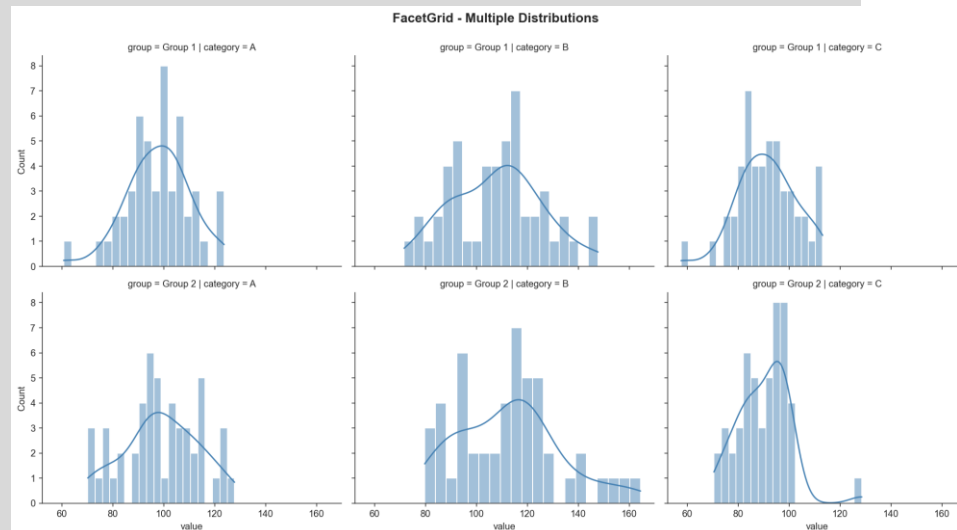
```
n = 300
```

```
data_facet = pd.DataFrame({  
    'value': np.concatenate([  
        np.random.normal(100, 15, n//3),  
        np.random.normal(110, 20, n//3),  
        np.random.normal(90, 10, n//3)  
    ]),  
    'category': np.repeat(['A', 'B', 'C'], n//3),  
    'group': np.tile(['Group 1', 'Group 2'], n//2)  
})
```

Create FacetGrid

```
g = sns.FacetGrid(data_facet, col='category', row='group', height=4, aspect=1.2)  
g.map(sns.histplot, 'value', bins=20, kde=True, color='steelblue')  
g.add_legend()  
g.fig.subplots_adjust(top=0.9)  
g.fig.suptitle('FacetGrid - Multiple Distributions', fontsize=14, fontweight='bold')
```

```
plt.show()
```



Interactive Visualization with Plotly

What is Plotly?

- Plotly creates interactive, publication-quality graphs
- Key Features
 - Interactive by default
 - Web-based visualizations
 - Wide variety of chart types
 - Export to HTML, PNG, PDF
- Installation
 - \$ pip install plotly
 - \$ conda install plotly



```
import plotly.graph_objects as go
import plotly.express as px
import pandas as pd
import numpy as np
```

Plotly Express vs Graph Objects

- Two Main APIs

Method 1: Plotly Express (High-level)

Quick and simple

```
df = px.data.iris()
```

```
fig = px.scatter(df, x='sepal_width', y='sepal_length', color='species', title='Iris Dataset – Express')
```

```
fig.show()
```

```
fig.write_html('iris_plot.html')
```

Method 2: Graph Objects (Low-level)

More control

```
fig = go.Figure()
```

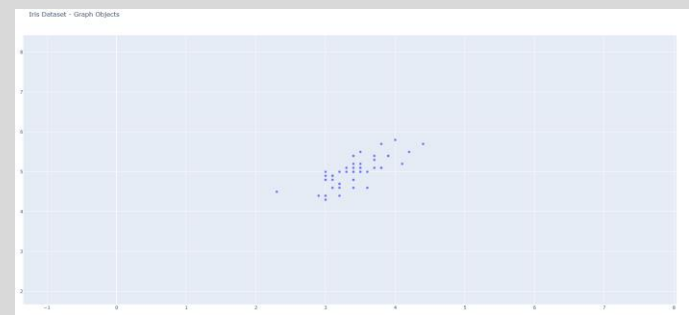
```
fig.add_trace(go.Scatter(  
    x=df[df['species']=='setosa']['sepal_width'],  
    y=df[df['species']=='setosa']['sepal_length'],  
    mode='markers',  
    name='setosa'
```

```
))
```

```
fig.update_layout(title='Iris Dataset - Graph Objects')
```

```
fig.show()
```

```
fig.write_html('iris_plot_2.html')
```



Plotly Express vs Graph Objects

- When to use each?
 - Plotly Express: Quick exploration
 - Graph Objects: Custom, complex visualizations

Interactive Line Chart

```
# Generate time series data
```

```
dates = pd.date_range('2024-01-01',  
periods=365, freq='D')  
np.random.seed(42)  
values1 = np.cumsum(np.random.randn(365))  
+ 100  
values2 = np.cumsum(np.random.randn(365))  
+ 95  
values3 = np.cumsum(np.random.randn(365))  
+ 105
```

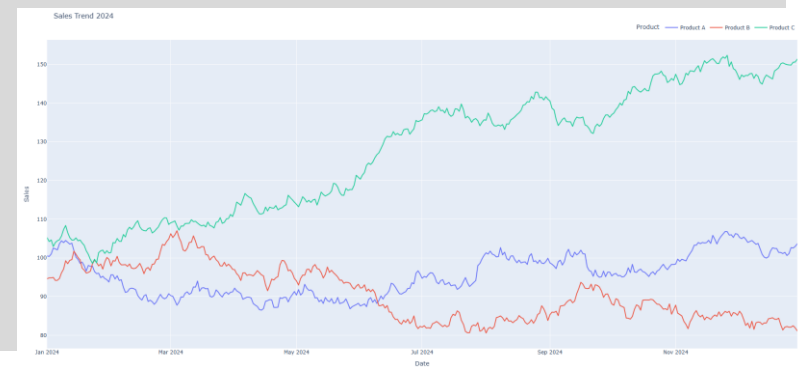
```
df_time = pd.DataFrame({  
    'Date': dates,  
    'Product A': values1,  
    'Product B': values2,  
    'Product C': values3  
})
```

```
# Create interactive line chart
```

```
fig = px.line(df_time, x='Date', y=['Product  
A', 'Product B', 'Product C'], title='Sales  
Trend 2024', labels={'value': 'Sales', 'variable':  
'Product'})
```

```
fig.update_layout( hovermode='x unified',  
font=dict(size=12), title_font_size=16,  
legend=dict(orientation='h', yanchor='bottom',  
y=1.02, xanchor='right', x=1) )
```

```
fig.show()
```



- Interactive Features: Zoom, pan, hover details

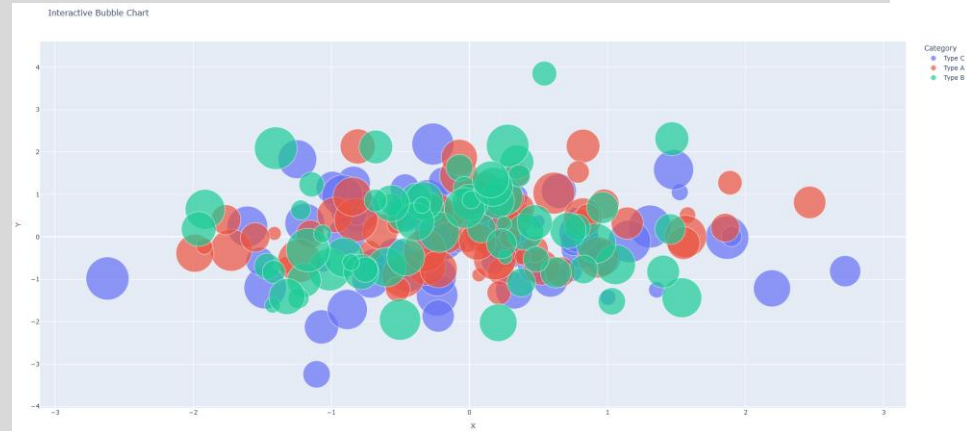
Interactive Scatter Plot

```
# Generate sample data
np.random.seed(42)
n = 200
df_scatter = pd.DataFrame({
    'X': np.random.randn(n),
    'Y': np.random.randn(n),
    'Size': np.random.randint(10, 100, n),
    'Category': np.random.choice(['Type A', 'Type B', 'Type C'], n),
    'Value': np.random.randint(50, 150, n)
})
```

```
# Create bubble chart
fig = px.scatter(df_scatter,
    x='X',
    y='Y',
    size='Size',
    color='Category',
    hover_data=['Value'],
    title='Interactive Bubble Chart', size_max=60)
```

```
fig.update_layout(
    font=dict(size=12),
    title_font_size=16,
    showlegend=True
)

fig.show()
```



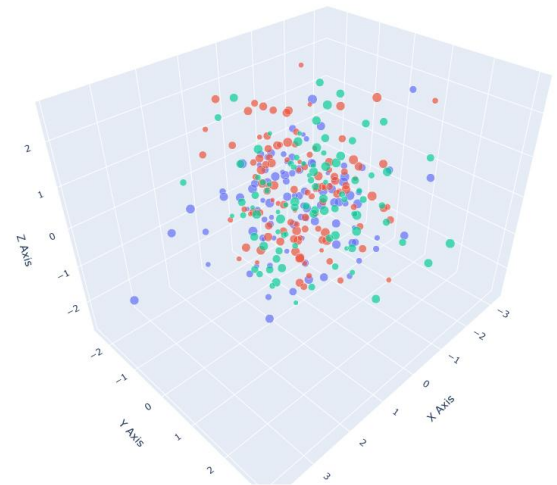
3D Scatter Plot

```
# Generate 3D data
np.random.seed(42)
n = 300
df_3d = pd.DataFrame({
    'X': np.random.randn(n),
    'Y': np.random.randn(n),
    'Z': np.random.randn(n),
    'Category': np.random.choice(['A', 'B', 'C'],
n),
    'Size': np.random.randint(5, 20, n)
})
```

```
# Create 3D scatter plot
fig = px.scatter_3d(df_3d, x='X', y='Y', z='Z',
color='Category', size='Size', title='3D Scatter
Plot', opacity=0.7)

fig.update_layout( scene=dict( xaxis_title='X
Axis', yaxis_title='Y Axis', zaxis_title='Z
Axis' ), font=dict(size=12),
title_font_size=16 )

fig.show()
```



- Interaction: Rotate, zoom, pan

Interactive Bar Chart

Sample data

```
categories = ['Q1', 'Q2', 'Q3', 'Q4']  
df_bar = pd.DataFrame({  
    'Quarter': categories * 3,  
    'Product': ['Product A']*4 + ['Product B']*4 + ['Product C']*4,  
    'Sales': [45, 52, 48, 60, 38, 42, 45, 50, 35, 40, 43, 48]  
})
```

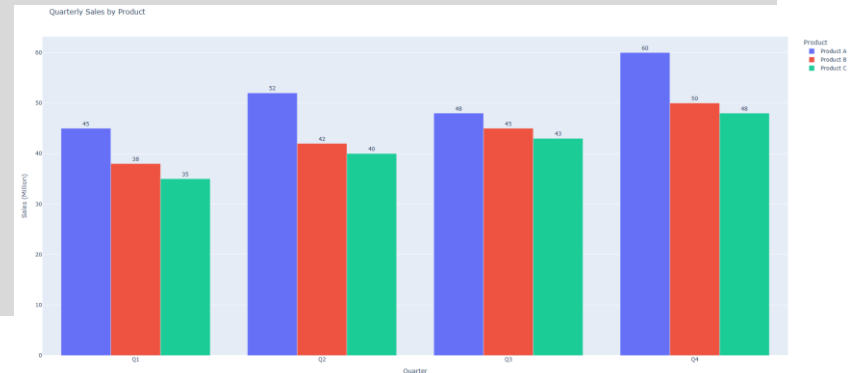
Create grouped bar chart

```
fig = px.bar(df_bar, x='Quarter', y='Sales', color='Product', barmode='group', title='Quarterly  
Sales by Product', text='Sales')
```

```
fig.update_traces(texttemplate='%{text}', textposition='outside')
```

```
fig.update_layout( font=dict(size=12), title_font_size=16, yaxis_title='Sales (Million)' )
```

```
fig.show()
```



Sunburst Chart

```
# Hierarchical data
df_sunburst = pd.DataFrame({
    'labels': ['Total', 'Asia', 'Europe', 'Americas', 'China', 'Japan', 'France', 'Germany',
    'USA', 'Brazil'],
    'parents': ['', 'Total', 'Total', 'Total', 'Asia', 'Asia', 'Europe', 'Europe', 'Americas',
    'Americas'],
    'values': [100, 45, 30, 25, 25, 20, 18, 12, 15, 10]
})

# Create sunburst chart
fig = px.sunburst(df_sunburst, names='labels', parents='parents', values='values',
title='Regional Sales Distribution')

fig.update_layout(
    font=dict(size=14),
    title_font_size=16
)

fig.show()
```



Animated Scatter Plot

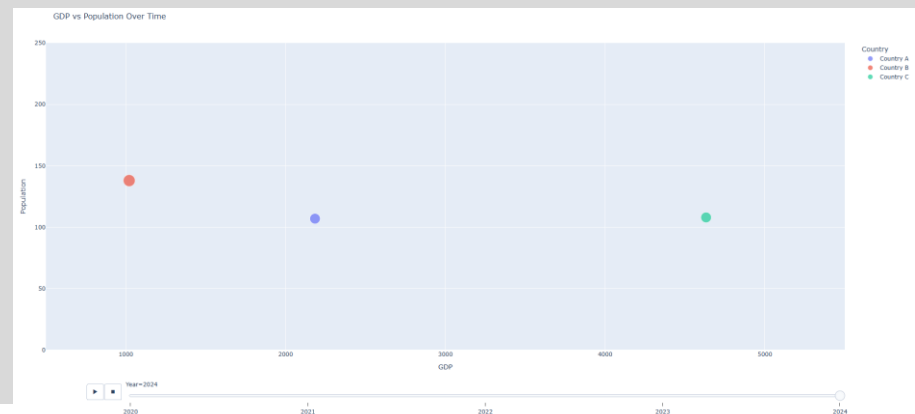
```
# Generate animated data
years = list(range(2020, 2025))
countries = ['Country A', 'Country B', 'Country C']
data_animated = []

np.random.seed(42)
for year in years:
    for country in countries:
        data_animated.append({
            'Year': year,
            'Country': country,
            'GDP': np.random.randint(1000, 5000),
            'Population': np.random.randint(50, 200)
        })
```

```
df_animated = pd.DataFrame(data_animated)
```

```
# Create animated scatter plot
fig = px.scatter(df_animated, x='GDP',
                y='Population', animation_frame='Year',
                animation_group='Country', size='Population',
                color='Country', hover_name='Country', title='GDP
                vs Population Over Time', range_x=[500, 5500],
                range_y=[0, 250])

fig.update_layout(
    font=dict(size=12),
    title_font_size=16
)
fig.show()
```



Geographic Maps

```
# Create sample geographic data
```

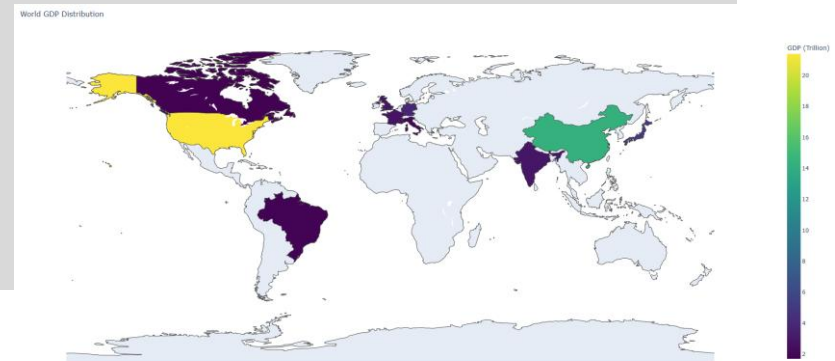
```
df_geo = pd.DataFrame({  
    'Country': ['USA', 'China', 'India', 'Japan', 'Germany', 'UK', 'France', 'Brazil', 'Italy', 'Canada'],  
    'GDP (Trillion)': [21.4, 14.3, 2.9, 5.1, 3.8, 2.8, 2.7, 1.8, 2.0, 1.7],  
    'Country Code': ['USA', 'CHN', 'IND', 'JPN', 'DEU', 'GBR', 'FRA', 'BRA', 'ITA', 'CAN']  
})
```

```
# Create choropleth map
```

```
fig = px.choropleth(df_geo, locations='Country Code', color='GDP (Trillion)',  
    hover_name='Country', color_continuous_scale='Viridis', title='World GDP Distribution')
```

```
fig.update_layout(  
    font=dict(size=12),  
    title_font_size=16,  
    geo=dict(showframe=False, showcoastlines=True)  
)
```

```
fig.show()
```



Advanced Plotly & Dashboards

Dashboard Layout

```
from plotly.subplots import make_subplots
```

```
# Create subplots
```

```
fig = make_subplots(
    rows=2, cols=2,
    subplot_titles=('Line Chart', 'Bar Chart', 'Scatter Plot', 'Pie Chart'),
    specs=[[{'type': 'scatter'}, {'type': 'bar'}], [{"type": 'scatter'}, {'type': 'pie'}]] )
```

```
# Add traces
```

```
# Line chart
```

```
x = np.linspace(0, 10, 100)
```

```
fig.add_trace(go.Scatter(x=x, y=np.sin(x), name='sin', line=dict(color='blue', width=2)), row=1, col=1)
```

```
# Bar chart
```

```
fig.add_trace(go.Bar(x=['A', 'B', 'C'], y=[10, 15, 13], name='Values', marker_color='lightgreen'), row=1, col=2)
```

```
# Scatter plot
```

```
fig.add_trace(go.Scatter(x=np.random.randn(50), y=np.random.randn(50), mode='markers',
    name='Data', marker=dict(size=10, color='coral')), row=2, col=1)
```

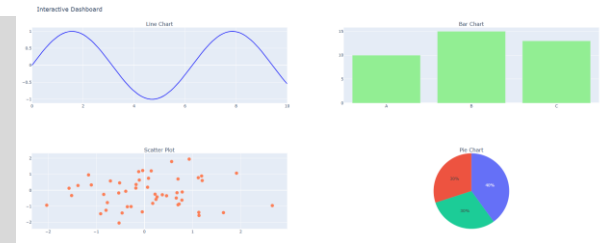
```
# Pie chart
```

```
fig.add_trace(go.Pie(labels=['Cat A', 'Cat B', 'Cat C'], values=[30, 40, 30]), row=2, col=2)
```

```
# Update layout
```

```
fig.update_layout( height=800, showlegend=False, title_text='Interactive Dashboard', title_font_size=18 )
```

```
fig.show()
```



Advanced Scatter Plots

- Scatter with Trend Lines

```
# Generate data with trend
```

```
np.random.seed(42)
```

```
x = np.random.randn(100)
```

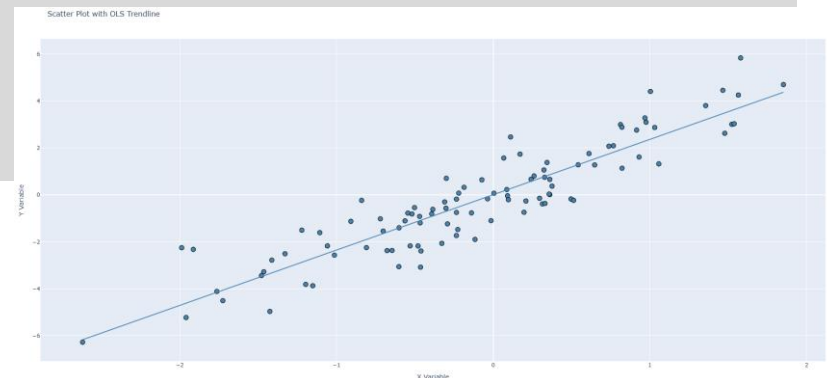
```
y = 2.5 * x + np.random.randn(100)
```

```
fig = px.scatter(x=x, y=y, trendline='ols', title='Scatter Plot with OLS Trendline',  
labels={'x': 'X Variable', 'y': 'Y Variable'})
```

```
fig.update_traces(marker=dict(size=10, color='steelblue', line=dict(width=1,  
color='black')))
```

```
fig.update_layout( font=dict(size=12), title_font_size=16, showlegend=True )
```

```
fig.show()
```



Candlestick Charts

```
# Generate stock price data
```

```
dates = pd.date_range('2024-01-01', periods=60, freq='D')  
np.random.seed(42)
```

```
open_price = 100 + np.cumsum(np.random.randn(60) * 2)  
high_price = open_price + np.random.rand(60) * 5  
low_price = open_price - np.random.rand(60) * 5  
close_price = open_price + np.random.randn(60) * 3
```

```
# Create candlestick chart
```

```
fig = go.Figure(data=[go.Candlestick( x=dates, open=open_price, high=high_price,  
low=low_price, close=close_price, name='Stock Price' )])
```

```
fig.update_layout( title='Stock Price Candlestick Chart', xaxis_title='Date',  
yaxis_title='Price', font=dict(size=12), title_font_size=16,  
xaxis_rangeslider_visible=False )
```

```
fig.show()
```



- Use case: Financial data visualization

Waterfall Charts

```
# Create waterfall data
fig = go.Figure(go.Waterfall(
    name="Financial Analysis", orientation="v",
    measure=["relative", "relative", "relative", "relative", "total", "relative", "relative",
"relative", "total"],
    x=["Revenue", "Cost", "Operating Exp", "Tax", "Net Q1", "Revenue", "Cost", "Operating
Exp", "Net Q2"],
    text=["+1000", "-400", "-200", "-100", "300", "+1200", "-450", "-250", "800"],
    y=[1000, -400, -200, -100, 0, 1200, -450, -250, 0],
    connector={"line": {"color": "rgb(63, 63, 63)"}},
))
```

```
fig.update_layout( title="Quarterly Financial Waterfall", showlegend=False,
font=dict(size=12), title_font_size=16, yaxis_title="Amount ($1000)" )
```

```
fig.show()
```



Funnel Charts

```
# Create funnel data
```

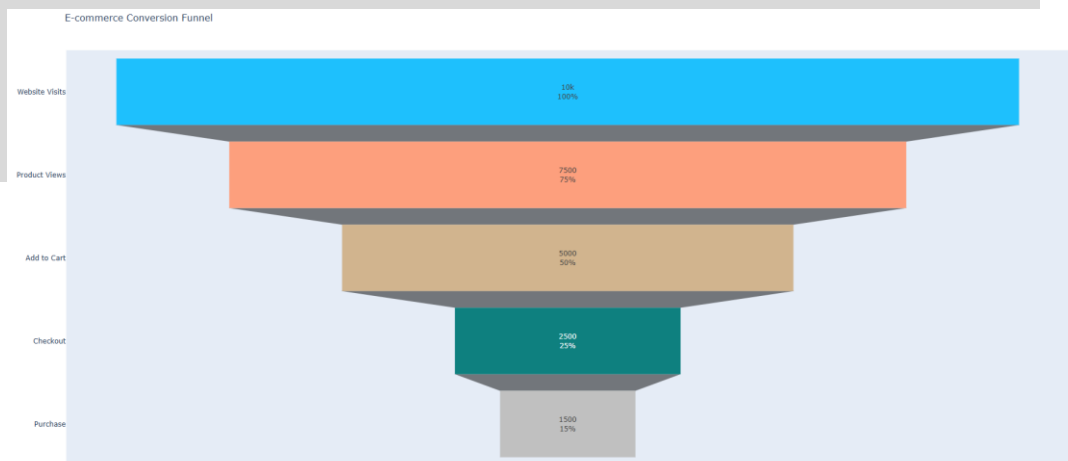
```
stages = ['Website Visits', 'Product Views', 'Add to Cart', 'Checkout', 'Purchase']
```

```
values = [10000, 7500, 5000, 2500, 1500]
```

```
fig = go.Figure(go.Funnel( y=stages, x=values, textposition="inside",  
textinfo="value+percent initial", marker={"color": ["deepskyblue", "lightsalmon",  
"tan", "teal", "silver"]} ))
```

```
fig.update_layout( title="E-commerce Conversion Funnel", font=dict(size=12),  
title_font_size=16 )
```

```
fig.show()
```

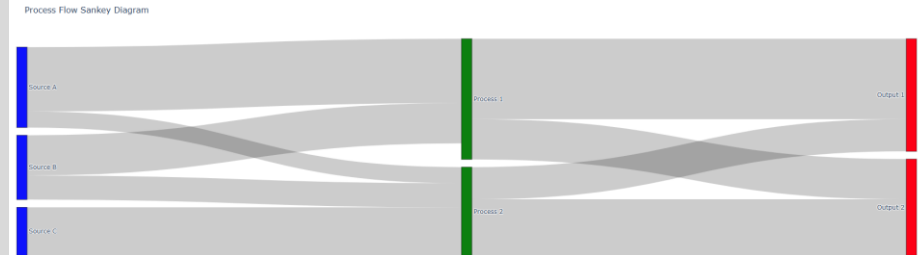


- Use case: Conversion analysis, sales pipeline

Sankey Diagrams

```
# Create Sankey diagram data
```

```
fig = go.Figure(data=[go.Sankey(  
    node=dict(  
        pad=15,  
        thickness=20,  
        line=dict(color="black", width=0.5),  
        label=["Source A", "Source B", "Source C", "Process 1", "Process 2", "Output 1", "Output 2"],  
        color=["blue", "blue", "blue", "green", "green", "red", "red"]  
    ),  
    link=dict(  
        source=[0, 0, 1, 1, 2, 3, 3, 4, 4],  
        target=[3, 4, 3, 4, 4, 5, 6, 5, 6],  
        value=[8, 2, 5, 3, 6, 10, 5, 4, 7]  
    )  
)])  
fig.update_layout( title="Process Flow Sankey Diagram", font=dict(size=12), title_font_size=16,  
height=600 )  
  
fig.show()
```



- Use case: Flow analysis, energy consumption

Interactive Heatmap with Plotly

```
# Generate correlation data
```

```
np.random.seed(42)
```

```
features = [f'Feature {i+1}' for i in range(10)]
```

```
correlation_matrix = np.random.randn(10, 10)
```

```
correlation_matrix = (correlation_matrix + correlation_matrix.T) / 2
```

```
np.fill_diagonal(correlation_matrix, 1)
```

```
# Create interactive heatmap
```

```
fig = go.Figure(data=go.Heatmap(
```

```
    z=correlation_matrix,
```

```
    x=features,
```

```
    y=features,
```

```
    colorscale='RdBu',
```

```
    zmid=0, text=np.round(correlation_matrix, 2),
```

```
    texttemplate='%{text}',
```

```
    textfont={"size": 10},
```

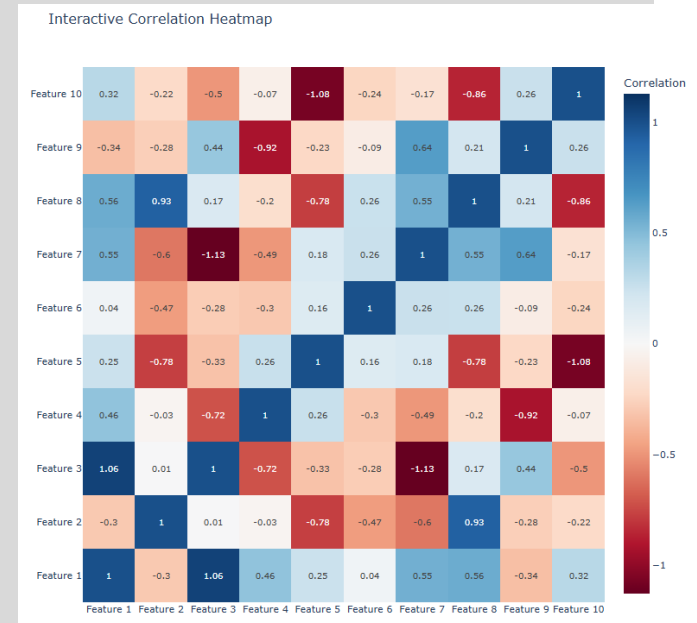
```
    colorbar=dict(title='Correlation')
```

```
))
```

```
fig.update_layout( title='Interactive Correlation Heatmap', font=dict(size=11), title_font_size=16,
```

```
width=800, height=800 )
```

```
fig.show()
```



Time Series with Range Slider

```
# Generate time series data
dates = pd.date_range('2020-01-01', '2024-12-31', freq='D')
np.random.seed(42)
values = 100 +
np.cumsum(np.random.randn(len(dates)) * 2)

# Create time series plot
fig = go.Figure()

fig.add_trace(go.Scatter(
    x=dates,
    y=values,
    mode='lines',
    name='Value',
    line=dict(color='steelblue', width=2)
))
```



```
# Add range slider
fig.update_xaxes(
    rangeslider_visible=True,
    rangeselector=dict(
        buttons=list([
            dict(count=1, label="1m", step="month",
                stepmode="backward"),
            dict(count=6, label="6m", step="month",
                stepmode="backward"),
            dict(count=1, label="1y", step="year",
                stepmode="backward"),
            dict(step="all", label="All")
        ])
    )
)

fig.update_layout( title='Time Series with
Range Selector', xaxis_title='Date',
yaxis_title='Value', font=dict(size=12),
title_font_size=16, height=600 )

fig.show()
```


Exporting Interactive Plots

```
# Create a sample plot
df = px.data.iris()
fig = px.scatter(df, x='sepal_width', y='sepal_length', color='species',
size='petal_length', title='Iris Dataset - Interactive')

# Export to HTML
fig.write_html('interactive_plot.html')
print("HTML file saved!")

# Export to static image
# Note: Requires kaleido package
# pip install kaleido --break-system-packages
# fig.write_image('plot.png', width=1200, height=800)
# fig.write_image('plot.pdf')

# Show in browser
fig.show()
```

Building a Complete Dashboard

```
# Create comprehensive dashboard
from plotly.subplots import make_subplots

# Generate sample data
np.random.seed(42)
dates = pd.date_range('2024-01-01', periods=12, freq='ME')
sales = np.random.randint(50, 150, 12)
profit = sales * np.random.uniform(0.15, 0.25, 12)
customers = np.random.randint(1000, 5000, 12)

# Create subplots with different types
fig = make_subplots(
    rows=3, cols=2,
    subplot_titles=('Monthly Sales', 'Profit Margin', 'Customer Growth', 'Product Mix', 'Regional
Performance', 'KPI Summary'),
    specs=[[{'type': 'scatter'}, {'type': 'bar'}], [{'type': 'scatter'}, {'type': 'pie'}], [{'type': 'bar'}, {'type':
'indicator'}]],
    vertical_spacing=0.12,
    horizontal_spacing=0.10
)
```

Building a Complete Dashboard

Row 1, Col 1: Monthly Sales

```
fig.add_trace(  
    go.Scatter(x=dates, y=sales, mode='lines+markers', name='Sales', line=dict(color='blue',  
width=3)), row=1, col=1  
)
```

Row 1, Col 2: Profit Margin

```
fig.add_trace(  
    go.Bar(x=dates, y=profit, name='Profit', marker_color='lightgreen'), row=1, col=2  
)
```

Row 2, Col 1: Customer Growth

```
fig.add_trace(  
    go.Scatter(x=dates, y=customers, mode='lines', fill='tozero', name='Customers',  
line=dict(color='coral')), row=2, col=1  
)
```

Row 2, Col 2: Product Mix

```
fig.add_trace(  
    go.Pie(labels=['Product A', 'Product B', 'Product C'], values=[35, 40, 25]), row=2, col=2  
)
```

Row 3, Col 1: Regional Performance

```
regions = ['North', 'South', 'East', 'West']  
regional_sales = [120, 95, 110, 88]
```

Building a Complete Dashboard

```
fig.add_trace(
    go.Bar(x=regions, y=regional_sales, name='Regional Sales', marker_color=['#FF6B6B',
'#4ECDC4', '#45B7D1', '#FFA07A']), row=3, col=1
)
# Row 3, Col 2: KPI Indicator
fig.add_trace(
    go.Indicator( mode="number+delta", value=sum(sales), delta={'reference': 1000, 'relative':
True}, title={'text': "Total Sales"}, number={'suffix': " units"}, domain={'x': [0, 1], 'y': [0,
1]} ), row=3, col=2
)
# Update layout
fig.update_layout(
    height=1200, showlegend=False, title_text='Business Intelligence Dashboard',
title_font_size=20, title_x=0.5
)
# Update axes
fig.update_xaxes(title_text="Date", row=1, col=1)
fig.update_xaxes(title_text="Date", row=1, col=2)
fig.update_xaxes(title_text="Date", row=2, col=1)
fig.update_xaxes(title_text="Region", row=3, col=1)
```

Building a Complete Dashboard

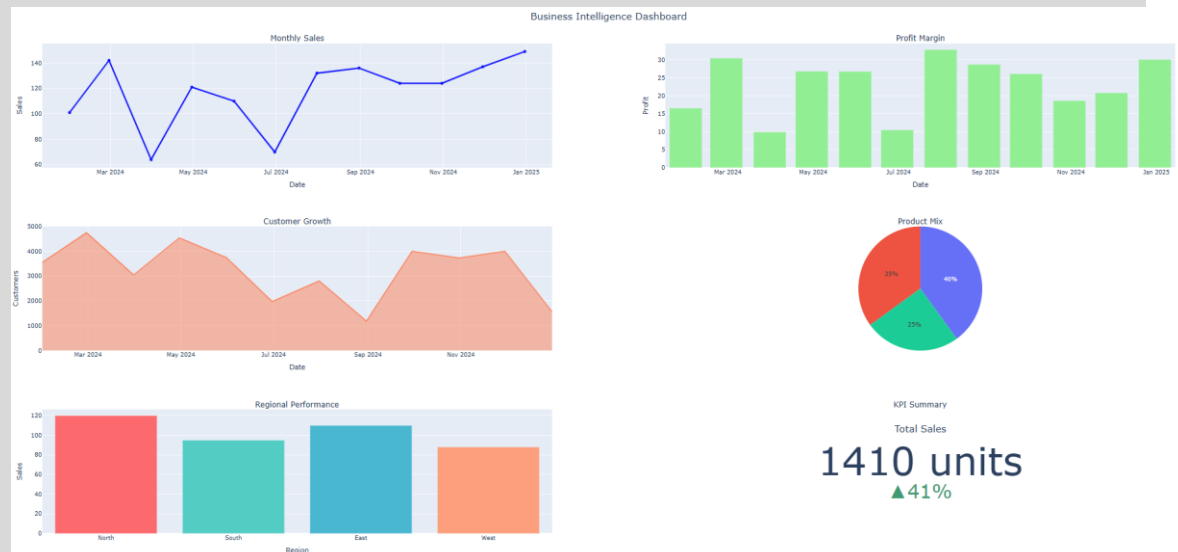
```
fig.update_yaxes(title_text="Sales", row=1, col=1)
fig.update_yaxes(title_text="Profit", row=1, col=2)
fig.update_yaxes(title_text="Customers", row=2, col=1)
fig.update_yaxes(title_text="Sales", row=3, col=1)
```

```
fig.show()
```

```
# Save dashboard
```

```
fig.write_html('business_dashboard.html')
```

```
print("Dashboard saved as 'business_dashboard.html'!")
```



When should I use Matplotlib vs Seaborn vs Plotly?

- Matplotlib: Maximum control, publication-quality, custom visualizations
- Seaborn: Statistical analysis, quick exploration, beautiful defaults
- Plotly: Interactive dashboards, web applications, presentations